



16575 SW 72nd Ave
Portland, OR 97224-7701 USA
800.329.2225 (+1.503.597.5350)
faxback.com | developer.support@faxback.com

Portal API Developer Reference

Last Edited February 18, 2021

Authors
Vin Delia
Derek Arduino
Quentin J. Dible
Mike Oliszewski

Abstract

Technical reference for the FaxBack Cloud Service API.

Table of Contents

Table of Contents	2
Overview	4
How To: Create an Account	4
How To: Send a Fax	4
How To: Receive a Fax.....	4
Notes.....	4
URLS	5
Session Management	6
Login	6
RefreshId	6
Logout	7
Account Management.....	8
ReadAccountGuids	8
Group Members	9
Logged-In Clients.....	9
ReadAccountBlock.....	10
CSV Response	12
Login Details	12
ReadAccount	13
AddAccount	14
ModifyAccount	15
DeleteAccount	16
GetGroupMemberships.....	17
Cover Template Management.....	19
Cover Replacement Fields	19
Template Properties.....	20
GetTemplates	21
GetTemplateContent.....	23
AddTemplate	24
AddTemplate via Form POST	24
DeleteTemplate	26
Email Alias Management	26
GetEmailAliases	26
CreateEmailAlias	27
DeleteEmailAlias	27
GetATAs	27
Group Management.....	30
ReadGroupGuids	30
ReadGroupBlock	30
ReadGroup	31
AddGroup	32
ModifyGroup	32

DeleteGroup	33
AddGroupMembers	33
RemoveGroupMembers	33
DID Management	35
ReadDIDBlock	35
ReadDIDGuids	36
AddDID	38
ModifyDID	39
DeleteDID	39
ReadDID	40
Receive Routing	42
Domain Management	43
GetDomain	43
CreateDomain	44
ModifyDomain	45
DeleteDomain	45
GetAccountDomains	46
GetDomainInfo	47
Messaging	49
SendMessage	49
Validation Test	52
HTML Form POST	54
ReadQueue	58
dbFax FaxReceive Targets (receive side)	61
dbFax FaxSend Targets (send side)	61
ReadMessageBlock	61
BuildFaxImage	65
Example of BuildFaxImage using HTTP GET	65
Example of BuildFaxImage using HTTP POST	66
ReadMessage	67
DeleteMessage	72
AbortMessage	72
GetQueueCounts	73
API Object References	74
Account Object References	74
QueueProfileXml Reference (full description and details)	77
DID Object References	82
Group Object References	82
API Tester	83
Features Supported	83
Main Toolbar	84
Request/Response Toolbar	84
Single Request	85
Multiple Request	86

Overview

How To: Create an Account

To create an account, first you must Login to receive your LoginId. This LoginId will be passed to each subsequent call. Once the LoginId has been retrieved, then you can call AddAccount and provide the properties for the account to be created. This function will return the unique identifier associated with the new account. After an account is created, the user will be able to send and/or receive faxes to this account.

How To: Send a Fax

To send a message, first you must Login to receive your LoginId. This LoginId will be passed to each subsequent call. Once the LoginId has been retrieved, then you can call SendMessage which returns a message handle. This message handle is an identifier for the message and is passed to any other function calls related to this message. To track the completion of SendMessage, you can call ReadMessage and pass it the message handle associated with the message of interest. After sending the message it is essential to call DeleteMessage with the message handle passed in to remove it from the database.

How To: Receive a Fax

To receive a fax, first you must Login to receive your LoginId. This LoginId will be passed to each subsequent call. Once the LoginId has been retrieved you can call ReadQueue to poll for new messages. This returns a list of message handles. Then, to view the content that was received you can download the message via BuildFaxImage by passing in the message handle associated with the message of interest. After downloading the message, it is essential to call DeleteMessage with the message handle passed in to remove it from the database.

Notes

All API functions in this document support both JSON and XML requests to the server. The data type can be specified in the request header or on the URL. Examples for how to structure each of these supported request types can be found in the examples for Login, AddAccount, and SendMessage. Links for testing each of the functions are provided, with URL information in {} indicating that the developer must provide this information on the URL to proceed. To test the API functions, the request URL in the request body of the API Tester must be updated with a valid server and parameters. The request URL is identified by "// URL:" in the request body. The  button will send the request to the server and the response will be displayed on the right in the response body viewer. When sending a request to an HTTP request URL from the API Tester, the API Tester must be loaded from an HTTP path. Conversely, when sending a request to an HTTPS request URL from the API Tester, the API Tester must be loaded from an HTTPS path.

URLS

Local server URL's area always formatted as follows:

<http://{nsx-server}:81/mqs/{service}/{method}>

Examples:

<http://localhost:81/mqs/nsx/Login>

<http://localhost:81/mqs/Accounts/AddAccount?LoginId={MyLoginId}>

<http://localhost:81/mqs/Groups/ReadGroupGuids?LoginId={MyLoginId}>

<http://localhost:81/mqs/Messages/SendMessage?LoginId={MyLoginId}>

NativeFax URL's are formatted similarly:

<https://api.nativefax.net/rest/{service}/{method}>

<https://sandbox.nativefax.net/rest/{service}/{method}>

Examples:

<http://api.nativefax.net/rest/nsx/Login>

<http://api.nativefax.net/rest/mqs/Accounts/AddAccount?LoginId={MyLoginId}>

<http://api.nativefax.net/rest/mqs/Groups/ReadGroupGuids?LoginId={MyLoginId}>

<http://api.nativefax.net/rest/Messages/SendMessage?LoginId={MyLoginId}>

Session Management

Login

The Login function establishes a session with the server using supplied user credentials.

Try it:

[Login - Plain Text Password](#)

[Login - PW2 Encoded Password](#)

Request(POST): <https://api.nativefax.net/rest/nsx/Login>
// HEADER: Content-Type: text/xml

```
<NSX>
  <Credentials>
    <UserName>AccountManagement</UserName>
    <Password>BlueSky27#</Password>
  </Credentials>
</NSX>
```

Response 400 Bad Request

Invalid input: Missing credentials (20578342/0x13A0026)

Response 401 Unauthorized

Invalid password (19988534/0x1310036)

Response 404 Not Found

User not found: AccountManagement (19988533/0x1310035)

Response 200 OK

```
<?xml version="1.0" encoding="utf-8"?>
<NSX>
  <LoginId>ea37be81-079a-4700-8265-c7eee3341c81</LoginId>
</NSX>
```

RefreshId

The RefreshId function is a lightweight means to keep a LoginId active at the server. A user login will time out if there is no activity for five minutes. No content needs to be sent to the server, just a LoginId in the URL, and no content is returned. The LoginId value to send for RefreshId is returned from the Login call.

If the response is "401 Unauthorized", a client should call Login again using cached credentials rather than reporting an error to the user. If a call to Login fails, then the user needs to be involved.

Try it:

Request(GET): https://api.nativefax.net/rest/nsx/RefreshId?LoginId={MyLoginId}
Response 400 Bad Request
Invalid input: Missing credentials (20578342/0x13A0026)
Response 401 Unauthorized
Invalid login id (19988536/0x1310038)
Response 204 OK

Logout

Logout a user login at the server.

Try it: [Logout](#)

Request(GET): https://api.nativefax.net/rest/nsx/Logout?LoginId={MyLoginId}
Response 400 Bad Request
Invalid input: Missing credentials (20578342/0x13A0026)
Response 204 No Content

Account Management

ReadAccountGuids

Returns a list of all Account Guids. If no group is specified in the request, then the response comes entirely from the accounts table without any group mapping applied to the query. Technically, this should be the same response as requesting all accounts from the "All users" group. However, by not specifying a group in the request the query will execute more quickly.

Try it: [ReadAccountGuids](#)

Request(GET): https://api.nativefax.net/rest/mqs/Accounts/ReadAccountGuids?LoginId={MyLoginId}
Response 400 Bad Request
Response 401 Unauthorized
Response 200 OK
<AccountGuids>097dfbe1-0cee-4653-9209-af6fc258675b,f745169e-2b53-461e-8376-65d723a0026d,e2ed456c-5125-4c43-b43b-13c591710ccb,d3c4a03e-ffa8-47d8-bdda-52209eeaa262,b6b917cd-9a28-4664-bb09-d81562a42362,8de0823a-835c-4f38-88aa-96cbf8f30111,7ab88cc9-35fb-4b97-b7b2-32b9afcb7c96,77062752-2661-4426-9605-c951738f60a9,6c8a429f-d099-4e91-bbe9-9d1ee981495b,fa960ae6-3af9-4d42-a054-de1f0b77d810</AccountGuids>

The response can be sorted by including an OrderBy query variable and specifying one or more columns, using SQL ORDER BY syntax:

```
http://{nsx-server}:81/mqs/nsx/Accounts/ReadAccountGuids?LoginId={MyLoginId}&OrderBy=Company%20ASC
```

The following list shows the columns names for accounts (subject to change!):

AccountGuid,AccountId,Description,Customer,Source,CDRInfo,CSID,TSID,ANI,SaveCDR,CallAhead,Client Logs,LoginGroup,RuleGroup,AllowSnd,AllowRcv,AllowSndOnBehalfOf,RcvFailoverGuid,RcvFailoverMode,ProvisioningArg,SerialNumber,MACAddress,EmailCoverType,UseCoverPage,CoverPage,BillingCode,ClientType,First,Middle,Last,Address1,Address2,City,State,PostalCode,Country,Company,ContactEmail,OfficePhone,CellPhone,QueueProfileXml,RawXml,DomainGuid,DomainId,CreatedOn,LastModified

Group Members

All accounts for a specific group can be returned by including a GroupId or GroupGuid variable in the request URL:

```
http://{nsx-server}:81/mqs/Accounts/ReadAccountGuids?LoginId={MyLoginId}&GroupGuid=AC785F88-C83F-4737-91D6-F64959BFA69A
```

Unless specified, all accounts will be returned from the domain to which the calling user belongs. If the calling user has access to multiple domains, then a DomainId or DomainGuid can be included in the request to specify the source domain.

```
http://{nsx-server}:81/mqs/Accounts/ReadAccountGuids?LoginId={MyLoginId}&DomainGuid=49B4A81D-C647-416B-A4FF-EF1D08A135EA
```

Logged-In Clients

Only the accounts that are logged-in, i.e., where account credentials have been used by a client to login to a Connection Server or to an NSX Server, can be returned by including Login=1 in the request URL.

```
http://{nsx-server}:81/mqs/Accounts/ReadAccountGuids?LoginId={MyLoginId}&Login=1
```

ReadAccountBlock

Returns account details for accounts associated with GUIDs returned from ReadAccountGuids. Specific account columns can be returned to limit the size of the response. Information about whether a client is logged-in using the account can be returned by adding a Login flag to the request.

Try it: [ReadAccountBlock](#)

Request(POST): <https://api.nativefax.net/rest/mqs/Accounts/ReadAccountBlock?LoginId={MyLoginId}>

```
<NSX>
  <AccountGuids>097dfbe1-0cee-4653-9209-af6fc258675b,f745169e-2b53-461e-8376-65d723a0026d</AccountGuids>
</NSX>
```

Response 400 Bad Request

Response 401 Unauthorized

Response 200 OK

```
<NSX>
  <Account>
    <AccountGuid>097dfbe1-0cee-4653-9209-af6fc258675b</AccountGuid>
    <AccountId>bridgedata</AccountId>
    <Description/>
    <Customer/>
    <Source>s1</Source>
    <CDRInfo/>
    <CSID/>
    <TSID/>
    <ANI/>
    <SaveCDR>0</SaveCDR>
    <CallAhead>0</CallAhead>
    <ClientLogs>0</ClientLogs>
    <LoginGroup/>
    <RuleGroup/>
    <AllowSnd>1</AllowSnd>
    <AllowRcv>1</AllowRcv>
    <AllowSndOnBehalfOf>0</AllowSndOnBehalfOf>
    <RcvFailoverGuid/>
    <RcvFailoverMode>0</RcvFailoverMode>
    <ProvisioningArg/>
    <SerialNumber/>
    <MACAddress/>
    <EmailCoverType>1</EmailCoverType>
    <UseCoverPage>0</UseCoverPage>
    <CoverPage/>
    <BillingCode/>
    <ClientType>100</ClientType>
    <First/>
    <Middle/>
    <Last/>
    <Address1/>
    <Address2/>
    <City/>
    <State/>
    <PostalCode/>
    <Country/>
    <Company/>
    <ContactEmail/>
    <OfficePhone/>
```

```

    <CellPhone/>
    <DomainGuid>981c6b8e-9718-402c-8ba2-904b71671eef</DomainGuid>
    <DomainId>Fax Server Domain</DomainId>
    <CreatedOn>2016-04-25 16:20:13Z</CreatedOn>
    <LastModified>2016-04-25 16:20:13Z</LastModified>
  </Account>
  <Account>
    <AccountGuid>f745169e-2b53-461e-8376-65d723a0026d</AccountGuid>
    <AccountId>Me2</AccountId>
    <Description/>
    <Customer/>
    <Source/>
    <CDRInfo/>
    <CSID/>
    <TSID/>
    <ANI/>
    <SaveCDR>0</SaveCDR>
    <CallAhead>0</CallAhead>
    <ClientLogs>0</ClientLogs>
    <LoginGroup/>
    <RuleGroup/>
    <AllowSnd>1</AllowSnd>
    <AllowRcv>1</AllowRcv>
    <AllowSndOnBehalfOf>0</AllowSndOnBehalfOf>
    <RcvFailoverGuid/>
    <RcvFailoverMode>0</RcvFailoverMode>
    <ProvisioningArg/>
    <SerialNumber/>
    <MACAddress/>
    <EmailCoverType>1</EmailCoverType>
    <UseCoverPage>0</UseCoverPage>
    <CoverPage/>
    <BillingCode/>
    <ClientType>0</ClientType>
    <First>My</First>
    <Middle/>
    <Last>Name</Last>
    <Address1/>
    <Address2/>
    <City/>
    <State/>
    <PostalCode/>
    <Country/>
    <Company/>
    <ContactEmail/>
    <OfficePhone/>
    <CellPhone/>
    <DomainGuid>981c6b8e-9718-402c-8ba2-904b71671eef</DomainGuid>
    <DomainId>Fax Server Domain</DomainId>
    <CreatedOn>2015-12-09 20:48:41Z</CreatedOn>
    <LastModified>2015-12-09 20:48:41Z</LastModified>
  </Account>
</NSX>

```

The return columns can be specified by putting an "Include" node in the request that contains a comma delimited list of column names:

Request(POST): <https://api.nativefax.net/rest/mqs/Accounts/ReadAccountBlock?LoginId={MyLoginId}>

```

<NSX>
  <AccountGuids>097dfbe1-0cee-4653-9209-af6fc258675b, f745169e-2b53-461e-8376-65d723a0026d</AccountGuids>
  <Include>AccountId,CreatedOn</Include>

```

</NSX>
Response 400 Bad Request
Response 401 Unauthorized
Response 200 OK
<pre> <NSX> <Account> <AccountId>bridgedata</AccountId> <CreatedOn>2016-04-25 16:20:13Z</CreatedOn> </Account> <Account> <AccountId>Me2</AccountId> <CreatedOn>2015-12-09 20:48:41Z</CreatedOn> </Account> </NSX> </pre>

CSV Response

The response can be in the form of a CSV file by including a query variable "As" with a value of "CSV" in the request. The response will consist of the first row comprised of column names followed by a row for each account record.

Request(GET): https://api.nativefax.net/rest/mqs/Accounts/ReadAccountBlock?LoginId={MyLoginId}&As=CSV
<pre> <NSX> <AccountGuids>097dfbe1-0cee-4653-9209-af6fc258675b</AccountGuids> </NSX> </pre>
Response 400 Bad Request
Response 401 Unauthorized
Response 200 OK
<pre> <NSX>AccountId, CreatedOn bridgedata, 2016-04-25 16:20:13Z Me2, 2015-12-09 20:48:41Z </NSX> </pre>

Login Details

Account login details can be returned by including Login=1 in the request URL. When present, account information can include the following (these fields can be included or excluded using the Include option to specify field names). If these fields have no value (examine the LoginId for a value) then the account is not being used to login a client anywhere.

LoginCreatedOn is the time of the initial login for the specified LoginId (the LoginId for IPFS clients is the *client instance*). LoginLastModified is the last time that the login was refreshed (the time when the client reestablished itself with the server to prevent being timed-out).

```
"ClientInfo": ";FV=9.2.8969.516;MN=QAWIN10PRO;LAN=10.0.0.132;IP=10.0.0.132",
"LoginSrvId": "Connection Server on QAWIN10PRO",
"LoginSrvGuid": "f196998a-a257-4af9-8f1b-0483a83d9ff8",
"LoginId": "a7037be4ceed43349fb30453792dc7ac",
"LoginCreatedOn": "2018-11-12 20:41:06Z",
"LoginLastModified": "2018-11-12 23:24:19Z"
```

Request(GET):

<http://{nsx-server}:81/mqs/Accounts/ReadAccountGuids?LoginId={MyLoginId}&Login=1>

Response 400 Bad Request**Response 401 Unauthorized****Response 200 OK**

<AccountGuids>f3b343a7-03b9-4cf7-a160-ade596a1ad3e,e326584c-e593-4445-884d-cc36e422d59d</AccountGuids>

ReadAccount

Read a single account from an AccountId or AccountGuid specified in the URL query, or the account associated with a DID that is specified in the URL query.

<http://{nsx-server}:81/mqs/Accounts/ReadAccount?LoginId={MyLoginId}&AccountId={MyAccountId}>

<https://api.nativefax.net/rest/mqs/Accounts/ReadAccount?LoginId={MyLoginId}&AccountId={MyAccountId}>

<http://{nsx-server}:81/mqs/Accounts/ReadAccount?LoginId={MyLoginId}&AccountGuid={MyGuid}>

<http://{nsx-server}:81/mqs/Accounts/ReadAccount?LoginId={MyLoginId}&DID={MyDID}>

<http://{nsx-server}:81/mqs/Accounts/ReadAccount?LoginId={MyLoginId}&MacAddress={MyMac}>

<http://{nsx-server}:81/mqs/Accounts/ReadAccount?LoginId={MyLoginId}&SerialNumber={MySN}>

<http://{nsx-server}:81/mqs/Accounts/ReadAccount?LoginId={MyLoginId}&SendEmailAddress={MySendI}>

Try it: [ReadAccount](#)

Request (POST)

```
{
  "NSX": {}
}
```

AddAccount

Adds a new account. Works very similar to documented Management API for AddAccount (<http://api.faxback.com/accountmanagement/webframe.html#FBAccountManagementAPI~FaxBack.API.Management~AddAccount.html>).

Try it: [AddAccount](#)

The following example is for adding a Fax ATA Account with two lines.

Request(POST): <https://api.nativefax.net/rest/mqs/Accounts/AddAccount?LoginId={MyLoginId}>

```
<NSX>
  <Account>
    <AccountId>FaxATA-D013579876</AccountId>
    <PasswordStr>BlueSky27#</PasswordStr>
    <ClientType>1</ClientType>
    <Lines>
      <FaxLine>
        <DID>5057071234</DID>
        <AnswerIncoming>1</AnswerIncoming>
        <OverrideProfile>
          <delivery_ntf>1</delivery_ntf>
          <email_format>0</email_format>
          <header_strip_count>0</header_strip_count>
          <non_delivery_ntf>1</non_delivery_ntf>
          <timezone_id>Pacific*</timezone_id>
        </OverrideProfile>
        <LineNum>0</LineNum>
        <LineId>Line 1</LineId>
        <CSID>5057071234</CSID>
        <TSID>5057071234</TSID>
        <ANI>5057071234</ANI>
        <ReceiveClient>1</ReceiveClient>
      </FaxLine>
      <FaxLine>
        <DID>5057074321</DID>
        <AnswerIncoming>1</AnswerIncoming>
        <OverrideProfile>
          <delivery_ntf>1</delivery_ntf>
          <email_format>0</email_format>
          <header_strip_count>0</header_strip_count>
          <non_delivery_ntf>1</non_delivery_ntf>
          <timezone_id>Pacific*</timezone_id>
        </OverrideProfile>
        <LineNum>0</LineNum>
        <LineId>Line 1</LineId>
        <CSID>5057074321</CSID>
        <TSID>5057074321</TSID>
        <ANI>5057074321</ANI>
        <ReceiveClient>1</ReceiveClient>
      </FaxLine>
    </Lines>
    <AuthorizedSendEmailAddresses/>
    <AllowSend>True</AllowSend>
    <ATAProperties>
      <CallAhead>0</CallAhead>
      <SerialNumber>D013579876</SerialNumber>
      <ClientLogging>0</ClientLogging>
    </ATAProperties>
```

</Account> </NSX>
Response 400 Bad Request
Response 401 Unauthorized
Response 200 OK
<AccountGuid>758a0dff-f5f7-4e38-a96c-cbc1301890fa</AccountGuid>

Parameters	Purpose
UseCoverPage	
RcvFailoverMode	[Flags()] public enum IPFS_FAILOVER_MODE { Default = 0, Always = 1, GroupMember = 2, RequirePrimaryLoggedIn = 4 // Makes sense only if combined with "Always" }
RcvFailoverGuid	Specifies the GUID of an account or group that should act as the failover account should the account not be able to receive a real-time fax, or is configured to always failover. Providing a RcvFailoverGuid of 01000000-0000-0000-0000-000000000000 (note the 1), <i>explicitly</i> requests that RcvFailoverGuid for the account should be set to the global Transmission Group. Providing an empty RcvFailoverGuid (i.e., set to 00000000-0000-0000-0000-000000000000) or not providing one and the account type is ATA, E-mail, FAXability or MSFax <i>implicitly</i> requests that RcvFailoverGuid for the account be set to the global Transmission Group.

ModifyAccount

Modifies an existing account. Works very similar to documented Management API for ModifyAccount (<http://api.faxback.com/accountmanagement/webframe.html#FBAccountManagementAPI~FaxBack.API.Management~ModifyAccount.html>).

You must include an AccountGuid to specify the account to modify.

A supervisor can modify any account within their domain. A user can only modify their user account.

Only include values that you want to change. In the example below, only the account description is modified with a new value.

Try it: [ModifyAccount](#)

Request(POST): <https://api.nativefax.net/rest/mqs/Accounts/ModifyAccount?LoginId={MyLoginId}>

```
<NSX>
  <Account>
    <AccountGuid>097dfbe1-0cee-4653-9209-af6fc258675b</AccountGuid>
    <Description>My description</Description>
  </Account>
</NSX>
```

Parameters	Purpose
RcvFailoverGuid	Specifies the GUID of an account or group that should act as the failover account should the account not be able to receive a real-time fax or is configured to always failover. Providing a RcvFailoverGuid of 01000000-0000-0000-0000-000000000000 (note the 1) requests that RcvFailoverGuid for the account should be set to the global Transmission Group. Providing an empty RcvFailoverGuid (i.e., set to 00000000-0000-0000-0000-000000000000) requests that RcvFailoverGuid for the account be cleared, meaning no receive failover.

The following account properties can only be changed by a supervisor. If a user attempts to change these account properties an "access denied" error will result.

Account Property
AllowSnd
AllowRcv
ReceiveFailoverGuid

DeleteAccount

Deletes an existing account

Try it: [DeleteAccount](#)

Request(POST): <https://api.nativefax.net/rest/mqs/Accounts/DeleteAccount?LoginId={MyLoginId}>

```
<NSX>
  <Account>
    <AccountGuid>097dfbe1-0cee-4653-9209-af6fc258675b</AccountGuid>
  </Account>
</NSX>
```


Optional Parameters	Purpose
DeleteDIDs	If 1, DIDs associated with the deleted account are also deleted. By default, DIDs associated with a deleted account are unlinked from the account and remain in the NSX domain.

GetGroupMemberships

Gets a listing of all groups for which the account is a member.

Provide a GroupId or GroupGuid node in order limit the response to that one group as a means of testing whether an account is in a specific group.

Provide an "Include" node that contains a list of comma separated column names to limit the response to only the specified columns.

Try it: [GetGroupMembershipsRa](#)

Request(POST): https://api.nativefax.net/rest/mqs/Accounts/GetGroupMemberships?LoginId={MyLoginId}
<pre><NSX> <Include>GroupId</Include> </NSX></pre>
Response 400 Bad Request
Response 401 Unauthorized
Response 200 OK
<pre><NSX> <Group> <GroupId>All Users</GroupId> </Group> <Group> <GroupId>Supervisors</GroupId> </Group> </NSX></pre>

Include? as=csv in the URL to receive an INI style response.

Example INI response:

Response 200 OK
<pre><NSX>GroupId All Users Supervisors </NSX></pre>

For a supervisor to get the group memberships of another account use:

Request(POST): <https://api.nativefax.net/rest/mqs/Accounts/GetGroupMemberships?LoginId={MyLoginId}>

```
<NSX>
  <Account>
    <AccountGuid>[Guid of other account]</AccountGuid>
  </Account>
  <Include>GroupGuid,GroupId</Include>
</NSX>
```

Cover Template Management

The following section describes the API functions for managing fax cover pages for a user. A supervisor account for a domain can be used to manage any users cover page templates otherwise the account is only allowed to manage its own covers. If a cover page is not specified for a user, then the domain supervisor account will be used to determine the cover template used.

NOTE: There is no modify template function. Instead you use AddTemplate with the property FailIfExists set to false.

The Administration management dialog for cover pages:

Edit Account: AccountManagement

Cover Page Settings

Default Template:

☒ Enable use of default template

Template Name
CoverPage.rtf

Cover Replacement Fields

The following replacement fields are supported for cover pages:

Parameter	Meaning
\$(SubmitTime)	Submission Time.
\$(Date)	Submission Date.

\$(Cover)	Message body text.
\$(SenderName)	Sender name.
\$(SenderFax)	Senders fax phone number.
\$(SenderVoice)	Senders voice phone number.
\$(SenderCompany)	Senders company name.
\$(From)	Replacement string for the macro \$(SenderName) which will look for the name in the email address string.
\$(To)	Recipient name(s) – This will be replaced by a comma separated list of all e-mail destination that are not marked as CC or BCC.
\$(Cc)	Will be replaced with a comma separated list of all e-mail destinations that are marked as CC. If the current destination is marked as BCC, its e-mail address will be appended to the list in a form (bcc:) Line will be wrapped on word boundary.
\$(ReceiverCompany)	Recipient company name.
\$(ReceiverName)	Recipient name – This results in a single recipient name per cover page (the names of any other recipients will be omitted).
\$(Subject)	Subject of the message.

Template Properties

Cover templates are stored with the following properties associated with them:

Parameter	Meaning
TemplateGuid	Unique identifier for this cover template.
TemplateName	Name used to identify this cover template, typically the uploaded file name.
TemplateSize	Size in bytes of the cover template image.
TemplateHash	Hash for validating the integrity of the cover template.
DomainGuid	Unique identifier for the domain associated with the user account.
DomainId	Friendly name associated with the domain.
DataSource	In HA environments with user replication services enabled this is the replication source for this template.
CreatedOn	DateTime cover template was initially uploaded to the server.
LastModified	DateTime cover page was last updated.

GetTemplates

Gets a listing of all cover templates for a user, but not the cover page content itself. Unless specified, the templates are those that belong to the logged in user.

Try it: [GetTemplates – All Properties](#)

Request: GET <https://api.nativefax.net/rest/mqs/Accounts/GetTemplates?LoginId={MyLoginId}>

Header: Content-Type: text/xml

Response: 400 Bad Request

Response: 401 Unauthorized

Response: 200 OK

Header: Content-Type: text/xml

```
<?xml version="1.0" encoding="utf-8"?>
<NSX>
  <DefaultTemplate>MyCoverTest.rtf</DefaultTemplate>
  <DefaultEnabled>1</DefaultEnabled>
  <Templates>
    <Template>
      <TemplateGuid>8946dd1b-93a1-4efb-a8cf-8d1aa97ab579</TemplateGuid>
      <TemplateName>CoverPage.rtf</TemplateName>
      <TemplateSize>41965</TemplateSize>
      <TemplateHash>61972EB9F927BA299CAF964F9D1D4F1C</TemplateHash>
      <DomainGuid>2cd5477c-e8bd-4857-8db3-9de10082e3c8</DomainGuid>
      <DomainId>DevSandBox Domain</DomainId>
      <DataSource />
      <CreatedOn>2019-07-17 17:11:38Z</CreatedOn>
      <LastModified>2019-07-17 17:11:38Z</LastModified>
    </Template>
  </Templates>
</NSX>
```

To retrieve the list of templates with only specific properties returned add a Include property with a comma separated list of property names you wish to retrieve.

Try it: [GetTemplates – Specific Properties](#)

Request: POST <https://api.nativefax.net/rest/mqs/Accounts/GetTemplates?LoginId={MyLoginId}>

Header: Content-Type: text/xml

```
<?xml version="1.0" encoding="utf-8"?>
<NSX>
  <Include>TemplateName,TemplateGuid</Include>
</NSX>
```

Response: 200 OK

Header: Content-Type: text/xml

```
<?xml version="1.0" encoding="utf-8"?>
<NSX>
  <DefaultTemplate>MyCoverTemplate.rtf</DefaultTemplate>
  <DefaultEnabled>1</DefaultEnabled>
  <Templates>
    <Template>
      <TemplateName>Another Upload.txt</TemplateName>
      <TemplateGuid>2af27463-f24f-432b-9a30-747d746d0800</TemplateGuid>
    </Template>
    <Template>
      <TemplateName>fred.txt</TemplateName>
      <TemplateGuid>6846f685-ac6b-4a63-adb3-36f8598a3b3d</TemplateGuid>
    </Template>
    <Template>
      <TemplateName>MyCoverTemplate.rtf</TemplateName>
      <TemplateGuid>6981e9b9-4a25-4932-873c-330efa1ca396</TemplateGuid>
    </Template>
    <Template>
      <TemplateName>Quick.rtf</TemplateName>
      <TemplateGuid>cdbdddb1-d408-4ca7-8196-ed40dbdf2679</TemplateGuid>
    </Template>
  </Templates>
</NSX>
```

To retrieve the list of templates as a CSV add as=CSV to the request URL and provide the Include property with property name list.

Try it: [GetTemplates - CSV](#)

Request: POST <https://api.nativefax.net/rest/mqs/Accounts/GetTemplates?LoginId={MyLoginId}&as=CSV>

Header: Content-Type: text/xml

```
<?xml version="1.0" encoding="utf-8"?>
<NSX>
  <Include>TemplateName,TemplateGuid</Include>
</NSX>
```

Response: 200 OK

Header: Content-Type: text/xml

```
<?xml version="1.0" encoding="utf-8"?>
<NSX>
  <DefaultTemplate>MyCoverTemplate.rtf</DefaultTemplate>
  <DefaultEnabled>1</DefaultEnabled>
  <Templates>TemplateName,TemplateGuid
Another Upload.txt,2af27463-f24f-432b-9a30-747d746d0800
fred.txt,6846f685-ac6b-4a63-adb3-36f8598a3b3d
MyCoverTemplate.rtf,6981e9b9-4a25-4932-873c-330efa1ca396
Quick.rtf,cdbdddb1-d408-4ca7-8196-ed40dbdf2679
</Templates>
</NSX>
```

To specify a specific users account for which to retrieve templates for, add AccountGuid or AccountId for the desired user to the URL.

Try it: [GetTemplates – Specific User](#)

Request: GET

<https://api.nativefax.net/rest/mqs/Accounts/GetTemplates?LoginId={guid}&AccountGuid={guid}>

Header: Content-Type: text/xml

Response: 200 OK

Header: Content-Type: text/xml

```
<?xml version="1.0" encoding="utf-8"?>
<NSX>
  <DefaultTemplate>MyCoverTemplate.rtf</DefaultTemplate>
  <DefaultEnabled>1</DefaultEnabled>
  <Templates>
    <Template>
      <TemplateName>Another Upload.txt</TemplateName>
      <TemplateGuid>2af27463-f24f-432b-9a30-747d746d0800</TemplateGuid>
    </Template>
  </Templates>
</NSX>
```

GetTemplateContent

Download the content of a cover page template from the server. This function was designed to be compatible with a browser request to download and save a template from the server to a local hard drive.

Try it: [GetTemplateContent - Current User](#)
[GetTemplateContent – Specific User](#)

Request: GET

<https://api.nativefax.net/rest/mqs/Accounts/GetTemplateContent?LoginId={MyLoginId}&TemplateGuid={MyGuid}>

Response: 400 Bad Request

Response: 401 Unauthorized

Response: 200 OK

Header: Content-Type: text/rtf

```
{\rtf1\ansi\ansicpg1252\deff0{\fonttbl{\f0\fnil\fcharset0 Calibri;}{\f1\fnil\fcharset0 Lucida Console;}}
{\colortbl ;\red255\green0\blue0;\red0\green176\blue80;}
{\*\generator Msftedit 5.41.21.2510;}\viewkind4\uc1\pard\sa200\sl276\slmult1\lang9\f0\fs22 This is an RTF
file.\par
\cf1\fs32 This is an RTF file.\par
$(Cover)\cf0\fs16\par
\cf2\f1\fs48 This is an RTF file.\cf0\f0\fs22\par
}
```

AddTemplate

Upload cover page template as a base64 string.

Try it: [AddTemplate – Current User](#)
[AddTemplate – Specific User](#)

Request: POST https://api.nativefax.net/rest/mqs/Accounts/AddTemplateContent?LoginId={MyLoginId}
Header: Content-Type: text/xml
<pre><?xml version="1.0" encoding="utf-8"?> <NSX> <Template> <TemplateName>Test.rtf</TemplateName> <ContentData>VGhpcyBpcyBhIHRlc3Qu</ContentData> <FailIfExists>False</FailIfExists> <DefaultTemplate>True</DefaultTemplate> </Template> </NSX></pre>
Response: 400 Bad Request
Response: 401 Unauthorized
Response: 200 OK HEADER: Content-Type: text/xml; charset=utf-8
<pre><?xml version="1.0" encoding="utf-8"?> <TemplateGuid>02237e97-15aa-45ca-b331-9ae6a1433878</TemplateGuid></pre>

AddTemplate via Form POST

Adds a new cover template or replaces an existing cover template by the same name, belonging to the specified account. This API call is handled as a posted form.

The form can optionally include a response HTML body supplied in a field of the form, which is used to return a structured response to the caller after the form is posted. If no response body is supplied, then the response is a comma separated string: <CoverTemplate Guid>,<Error Number>,<Error Text>.

Parameters	Purpose
DefaultTemplate	If 1, set this uploaded template as the account's default template.
Document	This is the file upload image.
FailIfExists	If 1, don't replace an existing template by the same name. If a template by the same name already exists for the account, an error is returned.
Password	Account password.
ResponseBody	A small amount of HTML with placeholders that defines the response content.

ResponseBodyBase64	You can specify where the server will put the cover template Guid, error number and error text in the response. The placeholders are: \$(Id) Replaced with the cover template Guid \$(ErrNum) Replaced with the response error number \$(ErrStr) Replaced with the response error text For example: <pre><html><body><pre>id=\$(Id)</pre><pre>errNum=\$(ErrNum)</pre><pre>errStr=\$(ErrStr)</pre></body></html></pre> Experience has shown that the ResponseBody should be supplied as a Base64 encoded version of the HTML (using ResponseBodyBase64), not straight HTML. Some browsers don't provide the MIME content correctly to the server when HTML is supplied as a form variable value.
TemplateName	Name of template to be displayed when referencing this template.
UserName	Account to add this cover page template to.

Try it: [AddTemplate via Web Form](#)

Example HTML test page:

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <title>Add Cover Template Form</title>
  </head>
  <body>
    <form name="TemplateForm"
      action="https://api.nativefax.net/rest/mqs/Accounts/AddTemplate" method="post"
      enctype="multipart/form-data">
      <table>
        <tr>
          <td>UserName:</td>
          <td><input name="UserName" type="text" size="40" value="AccountManagement" /></td>
        </tr>
        <tr>
          <td>Password:</td>
          <td><input name="Password" type="password" size="40" value="B1ueSky27#" /></td>
        </tr>
        <tr>
          <td>TemplateName:</td>
          <td><input id="TemplateName" type="text" size="40" /></td>
        </tr>
      </table>
      <input name="TemplateName" type="hidden" value="" />
    </form>
  </body>
</html>
```

```

<p><input name="Document" type="file" style="width:500px"/></p>
<p><input type="submit" value="Submit" /></p>
</form>
</body>
</html>

```

The default response for success with no HTML response body provided in the request:

```
69F38EBE-FF3F-4DFB-87A4-B4A26849B908,0,Ok
```

The default response for failure with no HTML response body provided in the request:

```
,20578361,Cover page template already exists
```

DeleteTemplate

Deletes an existing cover page template. The cover template can be specified by name, or by the template's TemplateGuid.

Try it: [DeleteTemplate – Current User](#)
[DeleteTemplate – Specific User](#)

Request: GET https://api.nativefax.net/rest/mqs/Accounts/DeleteTemplate?LoginId={MyLoginId}&TemplateGuid={guid}
Response: 400 Bad Request
Response: 401 Unauthorized
Response: 204 No Content

Email Alias Management

GetEmailAliases

Get a list of all email aliases for an account.

Optional Parameters	Purpose
Include	Provide an "Include" node to limit the columns that are returned. All columns include: EmailAlias, AccountGuid, AccountId, RawXml, DomainGuid, DomainId, CreatedOn, LastModified

Try it: [GetEmailAliases](#)

Request(POST): <https://api.nativefax.net/rest/mqs/Accounts/GetEmailAliases?LoginId={MyLoginId}>

```
<NSX>
  <Include>EmailAlias</Include>
</NSX>
```

Response 400 Bad Request

Response 401 Unauthorized

Response 200 OK

```
<NSX>
  <EmailAliasRecord>
    <EmailAlias>fred@whatever</EmailAlias>
  </EmailAliasRecord>
</NSX>
```

CreateEmailAlias

Try it: [CreateEmailAlias](#)

Request(POST): <https://api.nativefax.net/rest/mqs/Accounts/CreateEmailAlias?LoginId={MyLoginId}>

```
<NSX>
  <Account>
    <EmailAlias>fred@wilmaco.com</EmailAlias>
  </Account>
</NSX>
```

DeleteEmailAlias

Try it: [DeleteEmailAlias](#)

Request(POST): <https://api.nativefax.net/rest/mqs/Accounts/DeleteEmailAlias?LoginId={MyLoginId}>

```
<NSX>
  <Account>
    <EmailAlias>fred@wilmaco.com</EmailAlias>
  </Account>
</NSX>
```

RESPONSE: 400 Bad Request

RESPONSE 401 Unauthorized

RESPONSE 204 No Content

GetATAs

Gets a list of all ATAs associated with the users of an NSX Domain. The list can be ATAs that are *assigned*

Proprietary and Confidential, FaxBack, Inc. 2020

to NSX users, or those that are *unassigned*.

NOTE: The ATAs come from the Provisioning Server ATAs table, and NSX must be configured to know where to find this table. The Provisioning Server provides all functionality for managing ATAs.

Optional Parameters	Purpose
CSV	Provide a "CSV" node to get back an INI style response, for example: "CSV": "1" Optionally, include ?As=CSV in the request URL.
Include	Provide an "Include" node to limit the columns that are returned, for example: "Include": "SerialNumber,ProviderId" When getting unassigned ATAs, all columns include: SerialNumber, ProviderId, ProviderGuid, Status, Location, Description, CreatedOn, LastModified When getting assigned ATAs, all columns include: SerialNumber, ProviderId, ProviderGuid, Status, Location, Description, CreatedOn, LastModified, AccountGuid, AccountId
OrderBy	Provide an "OrderBy" node to sort the response according to the values from one of the queried columns, for example: "OrderBy": "ProviderId,SerialNumber"
Unassigned	Provide an "Unassigned" node to request ATAs that are not assigned to any user in the NSX Domain. For example: "Unassigned": "1" Optionally, include ?Unassigned=1 in the request URL.

Try it: [GetATAs](#)

Request(POST): <https://api.nativefax.net/rest/mqs/Accounts/GetATAs?LoginId={MyLoginId}>

```
<NSX>
  <OrderBy>ProviderId</OrderBy>
</NSX>
```

Response 400 Bad Request

Response 401 Unauthorized

Response 200 OK

```
<NSX>
  <ATAREcord>
    <SerialNumber>D08826438</SerialNumber>
    <ProviderId>MyProvider.provider</ProviderId>
    <ProviderGuid>19374251-4320-4364-8fb0-1445ce8ae4ce</ProviderGuid>
    <Status>0</Status>
    <Location>MyProvider</Location>
    <Description>MyProvider - 120 Shipped 10/7/2015</Description>
    <CreatedOn>2015-06-19 11:31:35Z</CreatedOn>
    <LastModified>2015-10-07 14:27:39Z</LastModified>
  </ATAREcord>
  <ATAREcord>
    <SerialNumber>D08046602</SerialNumber>
    <ProviderId>MyProvider.provider</ProviderId>
    <ProviderGuid>19374251-4320-4364-8fb0-1445ce8ae4ce</ProviderGuid>
    <Status>3</Status>
    <Location>MyProvider</Location>
    <Description>MyProvider - Shipped - 11/10/2015</Description>
    <CreatedOn>2015-06-18 15:17:43Z</CreatedOn>
    <LastModified>2015-11-10 11:50:59Z</LastModified>
  </ATAREcord>
  <ATAREcord>
    <SerialNumber>D08046515</SerialNumber>
    <ProviderId>MyProvider.provider</ProviderId>
    <ProviderGuid>19374251-4320-4364-8fb0-1445ce8ae4ce</ProviderGuid>
    <Status>3</Status>
    <Location>MyProvider</Location>
    <Description>MyProvider - Shipped - 11/10/2015</Description>
    <CreatedOn>2015-06-18 15:17:40Z</CreatedOn>
    <LastModified>2015-11-10 11:50:59Z</LastModified>
  </ATAREcord>
</NSX>
```

Group Management

ReadGroupGuids

Try it: [ReadGroupGuids](#)

Request(GET): <https://api.nativefax.net/rest/mqs/Groups/ReadGroupGuids?LoginId={MyLoginId}>

Response 200 OK

```
<GroupGuids>05ff6dd6-b4cd-4e02-a5c1-d0a44822b078,1e49bf32-7154-458e-ba2d-7fd2feffe515,81878b7e-ee94-4614-b2c6-409860ecbe06,cc859b5a-d5e8-450f-9c55-846e0173ed34</GroupGuids>
```

The response can be sorted by including an OrderBy query variable and specifying one or more columns, using SQL ORDER BY syntax:

```
http://{nsx-server}/mqs/nsx/Groups/ReadGroupGuids?OrderBy=Company%20ASC
```

The following list shows the columns names for accounts (subject to change!):

GroupGuid,GroupId,GroupType,MemberRouting,AllowRcv,RawXml,DomainGuid,DomainId,CreatedOn,LastModified

Unless specified, all groups will be returned from the domain to which the calling user belongs. If the calling user has access to multiple domains, then a DomainId or DomainGuid can be included in the request to specify the source domain.

```
http://{nsx-server}/mqs/Groups/ReadGroupGuids?DomainGuid=49B4A81D-C647-416B-A4FF-EF1D08A135EA
```

ReadGroupBlock

Try it: [ReadGroupBlock](#)

Request(POST): <https://api.nativefax.net/rest/mqs/Groups/ReadGroupBlock?LoginId={MyLoginId}>

```
<NSX>
  <GroupGuids>05ff6dd6-b4cd-4e02-a5c1-d0a44822b078,1e49bf32-7154-458e-ba2d-7fd2feffe515,81878b7e-ee94-4614-b2c6-409860ecbe06,cc859b5a-d5e8-450f-9c55-846e0173ed34</GroupGuids>
</NSX>
```

Response 200 OK

```
<NSX>
  <Group>
    <GroupGuid>05ff6dd6-b4cd-4e02-a5c1-d0a44822b078</GroupGuid>
    <GroupId>All Users</GroupId>
    <GroupType>0</GroupType>
```

```

    <MemberRouting>0</MemberRouting>
    <AllowRcv>1</AllowRcv>
    <DomainGuid>981c6b8e-9718-402c-8ba2-904b71671eef</DomainGuid>
    <DomainId/>
    <CreatedOn>2016-03-17 22:57:23Z</CreatedOn>
    <LastModified>2016-03-17 22:57:23Z</LastModified>
    <MembershipLastModified>2016-06-27 20:17:21Z</MembershipLastModified>
  </Group>
  <Group>
    <GroupGuid>1e49bf32-7154-458e-ba2d-7fd2feffe515</GroupGuid>
    <GroupId>Supervisors</GroupId>
    <GroupType>0</GroupType>
    <MemberRouting>0</MemberRouting>
    <AllowRcv>1</AllowRcv>
    <DomainGuid>981c6b8e-9718-402c-8ba2-904b71671eef</DomainGuid>
    <DomainId/>
    <CreatedOn>2016-03-17 22:57:24Z</CreatedOn>
    <LastModified>2016-03-17 22:57:24Z</LastModified>
    <MembershipLastModified>2016-06-24 22:33:43Z</MembershipLastModified>
  </Group>
  <Group>
    <GroupGuid>81878b7e-ee94-4614-b2c6-409860ecbe06</GroupGuid>
    <GroupId>System Users</GroupId>
    <GroupType>0</GroupType>
    <MemberRouting>0</MemberRouting>
    <AllowRcv>1</AllowRcv>
    <DomainGuid>981c6b8e-9718-402c-8ba2-904b71671eef</DomainGuid>
    <DomainId/>
    <CreatedOn>2016-04-18 17:58:44Z</CreatedOn>
    <LastModified>2016-04-18 17:58:44Z</LastModified>
    <MembershipLastModified>2016-05-12 15:44:28Z</MembershipLastModified>
  </Group>
  <Group>
    <GroupGuid>cc859b5a-d5e8-450f-9c55-846e0173ed34</GroupGuid>
    <GroupId>Postmasters</GroupId>
    <GroupType>0</GroupType>
    <MemberRouting>0</MemberRouting>
    <AllowRcv>1</AllowRcv>
    <DomainGuid>981c6b8e-9718-402c-8ba2-904b71671eef</DomainGuid>
    <DomainId/>
    <CreatedOn>2016-04-18 17:58:44Z</CreatedOn>
    <LastModified>2016-04-18 17:58:44Z</LastModified>
    <MembershipLastModified>2016-05-12 15:44:28Z</MembershipLastModified>
  </Group>
</NSX>

```

ReadGroup

Read a single group from an GroupId or GroupGuid specified in the URL query, or the group associated with a DID that is specified in the URL query.

Try it: [ReadGroup](#)

Example Request (credentials are required, but other input is in the URL query):

Request(GET):

<https://api.nativefax.net/rest/mqs/Groups/ReadGroup?LoginId={MyLoginId}&GroupId={MyGroupId}>

Request(GET):

<https://api.nativefax.net/rest/mqs/Groups/ReadGroup?LoginId={MyLoginId}&GroupGuid={MyGroupGuid}>

Request(GET):

<https://api.nativefax.net/rest/mqs/Groups/ReadGroup?LoginId={MyLoginId}&DID={MyDID}>

AddGroup

Adds a new group. Groups can be account groups or load-balance groups.

Try it: [AddGroup](#)

Parameter	Purpose
GroupId	The name of the group.
GroupType	
MemberRouting	For load-balance groups, specifies the strategy used for routing to group members. 0 = Alternating 1 = MostFree 2 = Random
AllowRcv	For load-balance groups, specifies if the group is enabled for receive.

Request(POST): <https://api.nativefax.net/rest/mqs/Groups/AddGroup?LoginId={MyLoginId}>

```
<NSX>
  <Group>
    <GroupId>My New Group</GroupId>
  </Group>
</NSX>
```

Response

```
<GroupGuid>cf2a701b-4513-4694-9422-6075161577da</GroupGuid>
```

ModifyGroup

Modifies an existing group.

Try it: [ModifyGroup](#)

Request(POST): <https://api.nativefax.net/rest/mqs/Groups/ModifyGroup?LoginId={MyLoginId}>

```
<NSX>
  <Group>
```



```
<GroupId>097dfbe1-0cee-4653-9209-af6fc258675b</GroupId>
<MemberRouting>0</MemberRouting>
</Group>
</NSX>
```

Response 204 No Content

DeleteGroup

Try it: [DeleteGroup](#)

Request(POST): <https://api.nativefax.net/rest/mqs/Groups/DeleteGroup?LoginId={MyLoginId}>

```
<NSX>
  <Group>
    <GroupGuid>097dfbe1-0cee-4653-9209-af6fc258675b</GroupGuid>
  </Group>
</NSX>
```

Optional Parameters	Purpose
DeleteDIDs	If 1, DIDs associated with the deleted group are also deleted. By default, DIDs associated with a deleted group are unlinked from the group and remain in the NSX domain.
FailIfMembers	If 1, an error will result if an attempt is made to delete a group that has members. By default, if a group has members when a group is deleted the members are removed from the group automatically.

AddGroupMembers

Adds one or more accounts to an existing group.

Try it: [AddGroupMembers](#)

Request(POST): <https://api.nativefax.net/rest/mqs/Groups/AddGroupMembers?LoginId={MyLoginId}>

```
<NSX>
  <Group>
    <GroupGuid>834B166D-7938-41B9-9EF9-8DD02EBC8528</GroupGuid>
    <AccountGuids>62D11CB4-BAAD-468D-A457-621B57C0F628,444CB3FF-3D98-4CB1-AD7F-7FF01C9352F4</AccountGuids>
  </Group>
</NSX>
```

RemoveGroupMembers

Removes one or more accounts from an existing group.

Try it: [RemoveGroupMembers](#)

Request(POST): <https://api.nativefax.net/rest/mqs/Groups/RemoveGroupMembers?LoginId={MyLoginId}>

```
<NSX>
  <Group>
    <GroupGuid>834B166D-7938-41B9-9EF9-8DD02EBC8528</GroupGuid>
    <AccountGuids>62D11CB4-BAAD-468D-A457-621B57C0F628,444CB3FF-3D98-4CB1-AD7F-7FF01C9352F4</AccountGuids>
  </Group>
</NSX>
```

DID Management

ReadDIDBlock

Read a group of DID records from a list of DIDGuids specified in the request.

Request(POST): [https://api.nativefax.net/rest/mqs/DIDs/ReadDIDBlock?LoginId=\\${MyLoginId}](https://api.nativefax.net/rest/mqs/DIDs/ReadDIDBlock?LoginId=${MyLoginId})

<DIDGuids>5b6fc1fc-4e5b-4a27-bde4-3ddb06e07d3f,787769f4-3c6b-4450-9cb8-991de7863fb3</DIDGuids>

Response

```
<?xml version="1.0" encoding="utf-8"?>
<NSX>
  <DID>
    <DIDGuid>5b6fc1fc-4e5b-4a27-bde4-3ddb06e07d3f</DIDGuid>
    <RouteToGuid>00000000-0000-0000-0000-000000000000</RouteToGuid>
    <RouteToId />
    <RouteToType>0</RouteToType>
    <DID>5057071234</DID>
    <DIDType>0</DIDType>
    <PIN />
    <PINMode>0</PINMode>
    <PINGuid>00000000-0000-0000-0000-000000000000</PINGuid>
    <Description />
    <Customer />
    <Source />
    <LineId>Line 1</LineId>
    <LineNum>0</LineNum>
    <TSID>5057071234</TSID>
    <CSID>5057071234</CSID>
    <ANI>5057071234</ANI>
    <RcvAnswer>1</RcvAnswer>
    <RcvFwdToGuid>00000000-0000-0000-0000-000000000000</RcvFwdToGuid>
    <BillingCode />
    <SndTransport />
    <QPXml/>
    <Xml/>
    <DomainGuid>2cd5477c-e8bd-4857-8db3-9de10082e3c8</DomainGuid>
    <DomainId>DevSandBox Domain</DomainId>
    <DataSource />
    <CreatedOn>2019-07-12 22:14:08Z</CreatedOn>
    <LastModified>2019-08-09 18:01:54Z</LastModified>
  </DID>
  <DID>
    <DIDGuid>787769f4-3c6b-4450-9cb8-991de7863fb3</DIDGuid>
    <RouteToGuid>00000000-0000-0000-0000-000000000000</RouteToGuid>
    <RouteToId />
    <RouteToType>0</RouteToType>
    <DID>5031234</DID>
    <DIDType>0</DIDType>
    <PIN />
    <PINMode>0</PINMode>
    <PINGuid>00000000-0000-0000-0000-000000000000</PINGuid>
```

```

<Description />
<Customer />
<Source />
<LineId />
<LineNum>0</LineNum>
<TSID />
<CSID />
<ANI />
<RcvAnswer>1</RcvAnswer>
<RcvFwdToGuid>00000000-0000-0000-0000-000000000000</RcvFwdToGuid>
<BillingCode />
<SndTransport />
<QPXml Rt="0" Ea="" Ef="0" Dn="0/8" Ndn="0/8" />
<DomainGuid>2cd5477c-e8bd-4857-8db3-9de10082e3c8</DomainGuid>
<DomainId>DevSandBox Domain</DomainId>
<DataSource />
<CreatedOn>2019-07-12 04:36:58Z</CreatedOn>
<LastModified>2019-07-12 04:37:04Z</LastModified>
</DID>
</NSX>

```

Try It: [ReadDIDBlock](#)

ReadDIDGuids

Reads every DID known to a domain, or DIDs specifically mapped to an account or group. Unless specified, the domain that is queried is the domain associated with the calling user.

Request(POST): [https://api.nativefax.net/rest/mqs/DIDs/ReadDIDGuids?LoginId=\\${MyLoginId}](https://api.nativefax.net/rest/mqs/DIDs/ReadDIDGuids?LoginId=${MyLoginId})

Response

```

<?xml version="1.0" encoding="utf-8"?>
<DIDGuids>5b6fc1fc-4e5b-4a27-bde4-3ddb06e07d3f,787769f4-3c6b-4450-9cb8-991de7863fb3,4c4fc922-1d62-4c34-8fee-dc7a5d877366</DIDGuids>

```

To request DIDs that are routed to a specific account or group, include AccountGuid in the URL.

Request(POST):
[https://api.nativefax.net/rest/mqs/DIDs/ReadDIDGuids?LoginId=\\${MyLoginId}&AccountGuid={AccountGuid}](https://api.nativefax.net/rest/mqs/DIDs/ReadDIDGuids?LoginId=${MyLoginId}&AccountGuid={AccountGuid})

Response

```

<?xml version="1.0" encoding="utf-8"?>
<DIDGuids>5b6fc1fc-4e5b-4a27-bde4-3ddb06e07d3f,4c4fc922-1d62-4c34-8fee-dc7a5d877366</DIDGuids>

```

To request only DIDs that are not assigned to any account or group, include Unassigned=1 in the URL.

Request(POST): https://api.nativefax.net/rest/mqs/DIDs/ReadDIDGuids?LoginId=\${MyLoginId}&Unassigned=1
Response
<?xml version="1.0" encoding="utf-8"?> <DIDGuids>787769f4-3c6b-4450-9cb8-991de7863fb3</DIDGuids>

The response can be sorted by including an OrderBy query variable and specifying one or more columns, using SQL ORDER BY syntax:

Request(POST): https://api.nativefax.net/rest/mqs/DIDs/ReadDIDGuids?LoginId=\${MyLoginId}&OrderBy=DID
Response
<?xml version="1.0" encoding="utf-8"?> <DIDGuids>787769f4-3c6b-4450-9cb8-991de7863fb3</DIDGuids>

The following list shows the column names for DIDs (subject to change):

DIDGuid,RouteToGuid,RouteToId,RouteToType,DID,Customer,Source,LineId,LineNum,TSID,CSID,ANI,RcvAnswer,RcvFwdToGuid,QueueProfileXml,RawXml,DomainGuid,DomainId,CreatedOn,LastModified

A topmost domain administrator can request a list of all DIDs from all domains. This function is for maintenance of the overall system, but it has a limited utility. For example, although you can get a list of all DIDs from all domains, DIDs can only be routed to a user/group when the DID and user/group are both in the same domain.

Request(POST): https://api.nativefax.net/rest/mqs/DIDs/ReadDIDGuids?LoginId=\${MyLoginId}&DomainId={DomainGUID}
Response
<?xml version="1.0" encoding="utf-8"?> <DIDGuids>787769f4-3c6b-4450-9cb8-991de7863fb3</DIDGuids>

AddDID

Creates a new DID in the inventory of DIDs. If a RoutedToGuid is included with the AddDID request, then the DID will also be assigned to the account or group specified by the RoutedToGuid.

Request(POST): [https://api.nativefax.net/rest/mqs/DIDs/AddDID?LoginId=\\${MyLoginId}](https://api.nativefax.net/rest/mqs/DIDs/AddDID?LoginId=${MyLoginId})

```
<NSX>
  <DID>
    <DID>5035550001</DID>
  </DID>
</NSX>
```

Response

```
<?xml version="1.0" encoding="utf-8"?>
<DIDGuid>d4c3d31b-acae-4203-b92a-5cfc51ca895c</DIDGuid>
```

Request(POST): [https://api.nativefax.net/rest/mqs/DIDs/AddDID?LoginId=\\${MyLoginId}](https://api.nativefax.net/rest/mqs/DIDs/AddDID?LoginId=${MyLoginId})

```
<NSX>
  <DID>
    <DID>5035550002</DID>
    <RouteToGuid>36b2be3b-b427-4255-88a9-809061b4b1bf</RouteToGuid>
  </DID>
</NSX>
```

Response

```
<?xml version="1.0" encoding="utf-8"?>
<DIDGuid>21d5a915-51bf-4fb9-9ce0-475d0fe587e2</DIDGuid>
```

Parameter	Purpose
SndTransport	An optional value that specifies the different types of transport methods that a message can be sent by. This value is applicable to faxes sent from ATAs, where the caller-id from the sending ATA is matched to a DID record. If SndTransport is empty or set to "*", then fax is the only transport. If set to any other value (one or more comma separated values), those values are considered transport ids. The dial number for the send session is processed by a transport plugin that is registered with NSX using the same transport id. For example, a value of "MaxMD" would specify that the ATA can only send faxes using the MaxMD transport method. If there are multiple values in this field and one of them is "*", for example, "*",MaxMD" then if none of the specified transport plugins accepted the dial number the fax can continue to be sent as a regular fax.

ModifyDID

Modifies an existing DID record. Within the DID object, the only required property is DIDGuid, which is used to specify the DID record being modified. The presence of additional properties dictates whether those properties are changed during the ModifyDID request.

In the following example, a DID is being returned to the pool of unassigned DIDs by including a RouteToGuid property with a value set to all zeros (00000000-0000-0000-0000-000000000000).

Request(POST): [https://api.nativefax.net/rest/mqs/DIDs/ModifyDID?LoginId=\\${MyLoginId}](https://api.nativefax.net/rest/mqs/DIDs/ModifyDID?LoginId=${MyLoginId})

```
<NSX>
  <DID>
    <DID>5035550001</DID>
    <RouteToGuid>00000000-0000-0000-0000-000000000000</RouteToGuid>
  </DID>
</NSX>
```

Response 204 No Content

DeleteDID

Deletes an existing DID. You can specify either the DID's GuidID, or the DID itself in the post.

Request(POST): [https://api.nativefax.net/rest/mqs/DIDs/DeleteDID?LoginId=\\${MyLoginId}](https://api.nativefax.net/rest/mqs/DIDs/DeleteDID?LoginId=${MyLoginId})

```
<NSX>
  <DID>
    <DID>5035550001</DID>
  </DID>
</NSX>
```

Response 204 No Content

Request(POST): [https://api.nativefax.net/rest/mqs/DIDs/DeleteDID?LoginId=\\${MyLoginId}](https://api.nativefax.net/rest/mqs/DIDs/DeleteDID?LoginId=${MyLoginId})

```
<NSX>
  <DID>
    <DIDGuid>21d5a915-51bf-4fb9-9ce0-475d0fe587e2</DIDGuid>
  </DID>
</NSX>
```

Response 204 No Content

ReadDID

Read a single DID record from a DID or DIDGuid specified in the URL query.

Request(POST):

[https://api.nativefax.net/rest/mqs/DIDs/ReadDID?LoginId=\\${MyLoginId}&DID=5035550001](https://api.nativefax.net/rest/mqs/DIDs/ReadDID?LoginId=${MyLoginId}&DID=5035550001)

Response

```
<?xml version="1.0" encoding="utf-8"?>
<DID>
  <DIDGuid>d4c3d31b-acae-4203-b92a-5cfc51ca895c</DIDGuid>
  <RouteToGuid>36b2be3b-b427-4255-88a9-809061b4b1bf</RouteToGuid>
  <RouteToId>MyAccount</RouteToId>
  <RouteToType>1</RouteToType>
  <DID>5035550001</DID>
  <DIDType>0</DIDType>
  <PIN />
  <PINMode>0</PINMode>
  <PINGuid>00000000-0000-0000-0000-000000000000</PINGuid>
  <Description />
  <Customer />
  <Source />
  <LineId />
  <LineNum>0</LineNum>
  <TSID />
  <CSID />
  <ANI />
  <RcvAnswer>1</RcvAnswer>
  <RcvFwdToGuid>00000000-0000-0000-0000-000000000000</RcvFwdToGuid>
  <BillingCode />
  <SndTransport />
  <QPXml/>
  <Xml/>
  <DomainGuid>2cd5477c-e8bd-4857-8db3-9de10082e3c8</DomainGuid>
  <DomainId>DevSandBox Domain</DomainId>
  <DataSource />
  <CreatedOn>2019-08-12 18:03:41Z</CreatedOn>
  <LastModified>2019-08-12 18:03:41Z</LastModified>
</DID>
```

Request(POST):

[https://api.nativefax.net/rest/mqs/DIDs/ReadDID?LoginId=\\${MyLoginId}&DIDGuid=21d5a915-51bf-4fb9-9ce0-475d0fe587e2](https://api.nativefax.net/rest/mqs/DIDs/ReadDID?LoginId=${MyLoginId}&DIDGuid=21d5a915-51bf-4fb9-9ce0-475d0fe587e2)

Response

```
<?xml version="1.0" encoding="utf-8"?>
<DID>
```



```

<DIDGuid>21d5a915-51bf-4fb9-9ce0-475d0fe587e2</DIDGuid>
<RouteToGuid>36b2be3b-b427-4255-88a9-809061b4b1bf</RouteToGuid>
<RouteToId>MyAccount</RouteToId>
<RouteToType>1</RouteToType>
<DID>5035550002</DID>
<DIDType>0</DIDType>
<PIN />
<PINMode>0</PINMode>
<PINGuid>00000000-0000-0000-0000-000000000000</PINGuid>
<Description />
<Customer />
<Source />
<LineId />
<LineNum>0</LineNum>
<TSID />
<CSID />
<ANI />
<RcvAnswer>1</RcvAnswer>
<RcvFwdToGuid>00000000-0000-0000-0000-000000000000</RcvFwdToGuid>
<BillingCode />
<SndTransport />
<QPXml/>
<Xml/>
<DomainGuid>2cd5477c-e8bd-4857-8db3-9de10082e3c8</DomainGuid>
<DomainId>DevSandBox Domain</DomainId>
<DataSource />
<CreatedOn>2019-08-12 18:03:41Z</CreatedOn>
<LastModified>2019-08-12 18:03:41Z</LastModified>
</DID>

```

Try it: [ReadDIDGuids/AddDID/ModifyDID/ReadDIDGuidsUser/DeleteDID](#)

Receive Routing

If the DID is being queried for receive routing purposes, then add "ReceiveRouting=1" in the request. The server will locate the specified DID, and then apply additional receive routing logic that includes:

- 1) Returning an NSX_MSG_CLASS.DID_DISABLED error if RcvAnswer mode is set to Never.
- 2) Following RcvFwdToGuid to a different DID record until a record is found where RcvFwdToGuid is not set (i.e., all zeros).
- 3) If there are multiple DIDs based on forwarding, keeping the QueueProfileXml from the first DID where the QueueProfileXml is set to override the account-level QueueProfileXml.

The response will include a Routing node in the returned DID that includes QueueProfileXml that is a made from a combination of the DID and account that the DID is mapped to. The routing QueueProfileXml will also contain any domain-level default properties.

Request(POST):

[https://api.nativefax.net/rest/mqs/DIDs/ReadDID?LoginId=\\${MyLoginId}&DID=5035550002](https://api.nativefax.net/rest/mqs/DIDs/ReadDID?LoginId=${MyLoginId}&DID=5035550002)

```
<NSX>
  <ReceiveRouting>1</ReceiveRouting>
  <Include>QueueProfileXml</Include>
  <AccountColumns>QueueProfileXml,AccountId,AccountGuid,DomainId,DomainGuid</AccountColumns>
</NSX>
```

Response

```
<?xml version="1.0" encoding="utf-8"?>
<DID>
  <QPXml/>
  <Routing>
    <QPXml/>
  </Routing>
  <Account>
    <QPXml/>
    <AccountId>MyAccount</AccountId>
    <AccountGuid>5225b30e-83a4-4902-b462-cb5271724336</AccountGuid>
    <DomainId>DevSandBox Domain</DomainId>
    <DomainGuid>2cd5477c-e8bd-4857-8db3-9de10082e3c8</DomainGuid>
  </Account>
</DID>
```

Try it: [ReadDID - ReceiveRouting](#)

Domain Management

GetDomain

The GetDomains function locates all NSX domains under a parent object in Object Directory. If the caller is a topmost domain supervisor, then the parent is the [Resources] folder in Object Directory. If the caller is a subdomain supervisor, then the parent is the domain of the caller, and the response includes the parent domain at the top of the list.

The function searches for all domains under Folder (type 2) and Domain (type 114) objects. The response is a hierarchical layout of all located domains.

Essentially, this will show the Domain information for the Domain the credentials you pass are a part of, as well as any subdomains under that Domain if there are any.

Request(POST): <https://api.nativefax.net/rest/nsx/GetDomains?LoginId={MyLoginId}>

Response

```
<?xml version="1.0" encoding="utf-8"?>
<NSX>
  <Domains>
    <Domain>
      <Id>DevSandBox Domain</Id>
      <Guid>2cd5477c-e8bd-4857-8db3-9de10082e3c8</Guid>
      <Type>114</Type>
    <Domain>
      <Id>TestDomain</Id>
      <Guid>f0f6cccf-18d4-406f-8173-591466809d65</Guid>
      <Type>114</Type>
    </Domain>
    <Domain>
      <Id>TestDomain2</Id>
      <Guid>dec3b025-b699-47c2-8789-6ccfa7d094b6</Guid>
      <Type>114</Type>
    </Domain>
  </Domains>
</NSX>
```

CreateDomain

The CreateDomain function creates a new NSX domain under a specified parent Object Directory Domain or Folder object. If no parent is specified, then the new domain will be created under the Object Directory [Resources] folder.

When a domain is created, NET SatisfAXtion default groups are created (All Users, Routers, Supervisors and System Users) and the groups are mapped to the new domain object with default rights appropriate for the group.

Optionally, additional users can be created when the domain is created, such as a default supervisor and/or postmaster user. Default supervisor and postmaster users are added to the appropriate group such that they have the rights that are appropriate for the type of user. However, all user names must be unique throughout all domains, so only one user can be named "Supervisor" or "Postmaster".

Request(POST): <https://api.nativefax.net/rest/nsx/CreateDomain?LoginId={MyLoginId}>

```
<NSX>
  <Credentials>
    <UserName>TestDomainAdmin</UserName>
    <Password>TestDomainPassword</Password>
  </Credentials>
  <ParentGuid> 2cd5477c-e8bd-4857-8db3-9de10082e3c8</ParentGuid>
  <DomainId>TestDomain</DomainId>
</NSX>
```

Response

```
<?xml version="1.0" encoding="utf-8"?>
<NSX>
  <DomainGuid>8f7b49d6-dd3b-4959-80ed-048bd0ef1aa4</DomainGuid>
</NSX>
```

ModifyDomain

The ModifyDomain function can rename a domain or change the license settings in the domain. Including nodes in the request for a setting will change that setting in the domain. To not change an existing value in the domain, do not include a node for that setting in the request.

Parameter	Meaning
DomainGuid -or- DomainIdToModify	Specify the domain to modify by GUID or name.
DomainId	Optional, if set the domain name is changed to the value of DomainId.

Request(POST): https://api.nativefax.net/rest/nsx/ModifyDomain?LoginId={MyLoginId}
<pre><NSX> <DomainGuid>fa18778b-7635-4d62-b2b1-1ed74ec26eca</DomainGuid> <DomainId>TestDomainModified</DomainId> </NSX></pre>
Response 204 No Content

DeleteDomain

The DeleteDomain function deletes the entire domain, including all users, groups and message queues associated with the domain.

Request(POST): https://api.nativefax.net/rest/nsx/DeleteDomain?LoginId=\${MyLoginId}
<pre><NSX> <DomainGuid> fa18778b-7635-4d62-b2b1-1ed74ec26eca </DomainGuid> </NSX></pre>
Response 204 No Content

Try it: [GetDomains](#) / [CreateDomain](#) / [ModifyDomain](#) / [DeleteDomain](#)

GetAccountDomains

Gets a list of domains that an account is in, and the rights that the account has within those domains.

Parameter	Meaning
AccountId -or- AccountGuid	Optional, a supervisor can specify another account to get the domains for that account.

In the list of returned domains, a domain with an attribute Primary=1 signifies that the account is created within that domain and faxing for that account will be managed within that domain. Where Primary=0, the account is mapped to that domain and will have typically supervisory rights to that domain.

Request(GET): <https://api.nativefax.net/rest/nsx/GetAccountDomains?LoginId={MyLoginId}>

Response

```
<?xml version="1.0" encoding="utf-8"?>
<NSX>
  <AccountId>AccountManagement</AccountId>
  <AccountGuid>78b93724-66c5-4984-9cd9-1a4888905cda</AccountGuid>
  <InTopmostDomain>0</InTopmostDomain>
  <Domains>
    <Domain Guid="f2216ffa-4266-4c11-905f-1ce9ea6c4db3" Id="TestDomain" Rights="PrivilegeUser,
PrivilegeRouter, PrivilegeSupervisor, PrivilegeSystemUser" Primary="0" />
    <Domain Guid="2cd5477c-e8bd-4857-8db3-9de10082e3c8" Id="DevSandBox Domain" Rights="PrivilegeUser,
PrivilegeRouter, PrivilegeSupervisor, PrivilegeSystemUser" Primary="1" />
  </Domains>
</NSX>
```

Request(POST): <https://api.nativefax.net/rest/nsx/GetAccountDomains?LoginId={MyLoginId}>

```
<NSX>
  <AccountId>User</AccountId>
</NSX>
```

Response 200 OK

```
<?xml version="1.0" encoding="utf-8"?>
<NSX>
  <AccountId>User</AccountId>
  <AccountGuid>18324342-a795-4d16-890f-3852f214cf03</AccountGuid>
  <InTopmostDomain>0</InTopmostDomain>
  <Domains>
    <Domain Guid="2cd5477c-e8bd-4857-8db3-9de10082e3c8" Id="DevSandBox Domain" Rights="PrivilegeUser"
Primary="1" />
  </Domains>
</NSX>
```

Try it: [GetAccountDomains](#)

GetDomainInfo

Validates that a domain exists and if requested returns queue counts for a specified domain.

Parameter	Meaning
QueueInfo	Optional, if set to 1 then the queue counts for the domain are returned.

Example request for GetDomainInfo that does not request QueueInfo:

Request(POST): https://api.nativefax.net/rest/nsx/GetDomaininfo?LoginId={MyLoginId}
<pre><NSX> <DomainId>DevSandBox Domain</DomainId> </NSX></pre>
Response 200 OK
<pre><?xml version="1.0" encoding="utf-8"?> <NSX> <DomainId>DevSandBox Domain</DomainId> <DomainGuid>2cd5477c-e8bd-4857-8db3-9de10082e3c8</DomainGuid> </NSX></pre>

Example request for GetDomainInfo that does request QueueInfo:

Request(POST): https://api.nativefax.net/rest/nsx/GetDomaininfo?LoginId={MyLoginId}
<pre><NSX> <DomainId>DevSandBox Domain</DomainId> <QueueInfo>1</QueueInfo> </NSX></pre>
Response 200 OK
<pre><?xml version="1.0" encoding="utf-8"?> <NSX> <DomainId>DevSandBox Domain</DomainId> <DomainGuid>2cd5477c-e8bd-4857-8db3-9de10082e3c8</DomainGuid> <TotalCounts> <Send>0</Send> <Sending>0</Sending> <Sent>33</Sent> <Receiving>0</Receiving> <Received>0</Received> <FaxEngine>Total from all fax engines</FaxEngine> </TotalCounts> <FaxEngines> <FaxEngine> <Send>0</Send> <Sending>0</Sending> <Sent>33</Sent></pre>

```
<Receiving>0</Receiving>
<Received>0</Received>
<FaxEngine>Default</FaxEngine>
</FaxEngine>
<FaxEngine>
  <Send>0</Send>
  <Sending>0</Sending>
  <Sent>0</Sent>
  <Receiving>0</Receiving>
  <Received>0</Received>
  <FaxEngine>File</FaxEngine>
</FaxEngine>
</FaxEngines>
</NSX>
```

Try it: [GetDomainInfo](#)

Messaging

SendMessage

Sends a message to one or more recipients. The message can be submitted as a form post or as Xml/Json.

Parameter	Purpose
ActionTypes	Optional, actions to be taken on completion. If omitted then the message will be deleted. Set to "ClientOutbox" to preserve the message in the users outbox.
AppInfo	Optional, a string value that is stored with the fax message that can be accessed later by an application for custom purposes.
CoverMode	Optional, specifies how a cover should be generated for the message. 0 = Use server configuration (it will be one of the following) 1 = No generated cover 2 = All recipients on cover page 3 = One recipient per cover page
CoverTemplate	Optional, the name of a cover page template to use when sending the fax. The name must match one of the user's cover page templates. If no name is supplied, then the user's default template is used. If the user does not have a default template, then the supervisor user's default template is used.
NoPageHeader	Optional, if set to 1 then the fax is sent without page headers. The default is 0 (add headers to pages). This option is intended for sending fax content that was recently created where each page contains an informative page header.
RecipientMode	Optional, specifies how the server handles validation errors on recipient fax numbers. Recipient fax numbers are examined by a validation process when SendMessage is called to detect malformed or blocked fax numbers. By default, the server will accept a message for delivery as long as there is one valid recipient. This behavior can be changed to require that all recipients are valid, so that if any recipient fails validation the message is rejected. 0 = Do not send if ALL fail validation (default) 1 = Do not send if ANY fail validation 2 = Test validation on recipients only, no message is sent.
ResponseBody ResponseBodyBase64	A small amount of HTML with placeholders that defines the response content. You can specify where the server will put the new message handle, error number and error text in the response. The placeholders are:

	\$(Id) Replaced with the message handle \$(ErrNum) Replaced with the response error number \$(ErrStr) Replaced with the response error text For example: <pre><html><body><pre>id=\$(Id)</pre><pre>errNum=\$(ErrNum)</pre><pre>errStr =\$(ErrStr)</pre></body></html></pre> Experience has shown that the ResponseBody should be supplied as a Base64 encoded version of the HTML (using ResponseBodyBase64), not straight HTML. Some browsers don't provide the MIME content correctly to the server when HTML is supplied as a form variable value.
Resolution	Optional, specifies the desired color depth (mono, grayscale, or color), DPI and page width of the fax. These following are the resolution enums. They are bits, so you can combine them. <pre>public enum RESOLUTION { HUFFMAN = 1, TWO_D_HUFFMAN = 2, T6 = 4 }</pre>
Schedule	Optional, a date and time in the future when the fax should be delivered. A scheduled fax will remain in the Send queue until it is eligible for sending. This value must be provided in UTC. For example: 2017-07-09 18:30:00Z
SenderCompany SenderName SenderFaxNumber SenderVoiceNumber Subject	Optional, values that are used to populate fields in cover page templates.
MessageBillingCode	Optional, a billing code value that is associated with this fax message that is eventually saved in a CDR following delivery.
RetryCountOverride	Optional, specifies the number of retries to attempt for the recipients of the message. -1 = No retries 0 = Use server default number of retries >0 = Number of retries

Recipient sections:

Parameter	Purpose
Address	The fax number to be dialed.
Name	Optionally sets a name associated with this recipient. The name is used for cover page creation.
Prefix	Optional, the type of recipient 0 = To 1 = Cc 2 = Bcc
LocalCSId	Optionally sets the Transmit Station Id (TSID) that a remote fax device sees when the fax is sent.
CallerId	Optionally sets the caller id (ANI) for the call when the fax is sent.

Document sections:

Parameter	Purpose
DocumentPart	Specifies how the document is to be treated. 0 = CoverMessage 1 = Document
DocumentType	Specifies the type of document being supplied. 0 = Unknown 1 = TIFF 2 = RTF 3 = PDF 4 = HTML 5 = TEXT
Name	Specifies the name of the document.
ContentText	Optional, for Xml/Json posts, specifies the text content of the document.
ContentData	Optional, for Xml/Json posts, specifies the content of the document using Base64 encoding.
PaperSize	Optional, specifies how the content should be converted to conform to a specified paper size, in particular the page length. The page width of the fax message is specified by the Resolution setting at the message level. The page width for an entire fax message must remain the same, and it is limited to widths that are supported by the receiving fax device (the most commonly supported width is 215mm that corresponds roughly to Letter or A4

	paper size). The page length is more flexible and can be unique for each page.
	Letter = 0, // Letter is default
	Legal = 1,
	GovernmentLetter = 2,
	GovernmentLegal = 3,
	Tabloid = 4,
	A3 = 5,
	A4 = 6,
	A5 = 7,
	B3 = 8,
	B4 = 9,
	B5 = 10,
	C3 = 11,
	C4 = 12,
	C5 = 13

Try it:

- [SendMessage - CoverOnly](#)
- [SendMessage - PDF](#)
- [SendMessage - HTML](#)

Request(POST): https://api.nativefax.net/rest/Messages/SendMessage?LoginId={MyLoginId}
<pre><NSX> <SendMessage> <Subject>subject of the message</Subject> <AppInfo>custom data</AppInfo> <Recipient> <Name>fred</Name> <Address>555-111-2222</Address> </Recipient> <Document> <DocumentPart>0</DocumentPart> <ContentText>This is the content for the cover page.</ContentText> </Document> </SendMessage> </NSX></pre>
Response 400 Bad Request
Response 401 Unauthorized
Response 200 OK
<pre><MessageHandle>S-9423d504-4e36-4b3c-bda3-b2e784cb8b04</MessageHandle></pre>

Validation Test

When SendMessage is called with RecipientMode set to 2 (ValidationTestOnly), all recipients are run through NSX validation and the results are returned without a fax message being submitted. The caller should not provide any content for the message as it will be ignored.

Example calling SendMessage using Json post to validate recipient fax numbers:

Request(POST): https://api.nativefax.net/rest/Messages/SendMessage?LoginId={MyLoginId}
<pre><NSX> <SendMessage> <RecipientMode>2</RecipientMode> <Recipient> <Address>12345</Address> </Recipient> <Recipient> <Address>9211</Address> </Recipient> </SendMessage> </NSX></pre>
Response 400 Bad Request
Response 401 Unauthorized
Response 200 OK
<pre><ValidationTest> <Status>Ok</Status> <StatusNum>0</StatusNum> <Recipient> <Address>12345</Address> <Status>Ok</Status> <StatusNum>0</StatusNum> </Recipient> <Recipient> <Address>9211</Address> <Status>Ok</Status> <StatusNum>0</StatusNum> </Recipient> </ValidationTest></pre>

Example validation test showing failure response:

Request(POST): https://api.nativefax.net/rest/Messages/SendMessage?LoginId={MyLoginId}
<pre><NSX> <SendMessage> <RecipientMode>2</RecipientMode> <Recipient> <Address>12345</Address> </Recipient> <Recipient> <Address>911</Address> </Recipient> </SendMessage> </NSX></pre>
Response 400 Bad Request
Response 401 Unauthorized
Response 200 OK
<pre><ValidationTest> <Status>Some recipients failed validation</Status></pre>

```

<StatusNum>20578354</StatusNum>
<Recipient>
  <Address>12345</Address>
  <Status>Ok</Status>
  <StatusNum>0</StatusNum>
</Recipient>
<Recipient>
  <Address>911</Address>
  <Status>Send was blocked by a validation rule</Status>
  <StatusNum>20578347</StatusNum>
</Recipient>
</ValidationTest>

```

HTML Form POST

With the HTML form (MIME) version of SendMessage, NSX server interprets all form-data as a series of node names/values that provide all necessary input to send a message. The node names in the form are the same names that NSX server looks for in a SendMessage request that was constructed using Xml/Json. Credentials can be passed in the uploaded MIME, or a LoginId can be provided in the URL.

Recipients and Documents are provided in *sections* where there is an obvious start and stop to the section. For example, the section starts with "**Recipient{**" with the brace suffix acting as a flag that means that the values that follow all belong to the same recipient, and stops with a value named "**}**" (closing brace) that means there are no more values for that recipient.

Example MIME showing a Recipient{ section:

```

Content-Disposition: form-data; name="Recipient{"

-----boundary123xyz
Content-Disposition: form-data; name="Name"

Fred
-----boundary123xyz
Content-Disposition: form-data; name="Address"

555-222-3333
-----boundary123xyz
Content-Disposition: form-data; name="}"

-----boundary123xyz

```

There are three ways to add content to a Document. The following examples show a complete Document{ } section, each with a different way to provide the content.

1. Provide file content in the Document{ form-data variable.

```

Content-Disposition: form-data; name="Document{"; filename="MyFax.tif"
Content-Type: image/tiff

II*.....(THE REST OF THE FILE)
-----Boundary1D2E3G4Q5B
Content-Disposition: form-data; name="DocumentPart"

1
-----Boundary1D2E3G4Q5B

```

```
Content-Disposition: form-data; name="}"
```

```
-----Boundary1D2E3G4Q5B
```

2. Provide file content anywhere in the uploaded MIME, and include a FileName form-data variable that references the file content by index using the format “[<index>]”, for example, “[0]”. An index of 0 references the first uploaded file in the MIME, an index of 1 references the second file, and so on.

```
Content-Disposition: form-data; name="Document123"; filename="MyFax.tif"
Content-Type: image/tiff
```

```
II*.....(THE REST OF THE FILE)
```

```
-----Boundary1D2E3G4Q5B
```

```
Content-Disposition: form-data; name="Document{"
```

```
-----Boundary1D2E3G4Q5B
```

```
Content-Disposition: form-data; name="DocumentPart"
```

```
1
```

```
-----Boundary1D2E3G4Q5B
```

```
Content-Disposition: form-data; name="FileName"
```

```
[0]
```

```
-----Boundary1D2E3G4Q5B
```

```
Content-Disposition: form-data; name="}"
```

```
-----Boundary1D2E3G4Q5B
```

3. Provide file content anywhere in the uploaded MIME, and include a FileName form-data variable that references the file content by form-data variable name.

```
Content-Disposition: form-data; name="DocumentXYZ"; filename="MyFax.tif"
Content-Type: image/tiff
```

```
II*.....(THE REST OF THE FILE)
```

```
-----Boundary1D2E3G4Q5B
```

```
Content-Disposition: form-data; name="Document{"
```

```
-----Boundary1D2E3G4Q5B
```

```
Content-Disposition: form-data; name="DocumentPart"
```

```
1
```

```
-----Boundary1D2E3G4Q5B
```

```
Content-Disposition: form-data; name="FileName"
```

```
DocumentXYZ
```

```
-----Boundary1D2E3G4Q5B
```

```
Content-Disposition: form-data; name="}"
```

```
-----Boundary1D2E3G4Q5B
```

The response to the MIME version of SendMessage is a small, formatted text body that includes a message handle, error number and error message.

```
<message handle>,<error number, 0 for success>,<error message>
```

Example response to SendMessage where the message was submitted successfully:

```
HTTP/1.1 200 OK
Server: NSX 9.2.8904.525
Date: Fri, 30 Jun 2017 18:55:40 GMT
Content-Length: 43
Content-Type: text/html

S-ecb9f0da-0221-414d-872b-3b3edca5ac88,0,Ok
```

Example response to SendMessage where the message failed to be submitted:

```
HTTP/1.1 200 OK
Server: NSX 9.2.8904.525
Date: Fri, 30 Jun 2017 18:55:40 GMT
Content-Length: 43
Content-Type: text/html

,20578353,All recipients failed validation
```

The MIME version of SendMessage was originally developed for web browser clients that expected successful and failed message submissions alike to have a "200 OK" response. To request that errors return an error code in the HTTP headers, add "NonBrowser=1" as a query variable to the URL. For example:

```
http://{nsx-server}/mqs/nsx/Messages/SendMessage?NonBrowser=1
```

Example response to SendMessage, using NonBrowser=1, where the message failed to be submitted:

```
400 Bad Request
Server: NSX 9.2.8904.525
Content-Length: 55
Content-Type: text/plain
Date: Wed, 19 Jul 2017 19:06:17 GMT

All recipients failed validation (20578353/0x13A0031)
```

Try it:

[SendMessage - HTML](#)

Example calling SendMessage using form post from HTML:

Request

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <title>Add Cover Template Form</title>
    <style>.label { width:90px; white-space:nowrap; }</style>
  </head>
  <body>
    <form name="SendFax" action="https://api.nativefax.net/rest/mqs/Messages/SendMessage"
```



```

        method="post" enctype="multipart/form-data">
<table style="width:100%">
  <tr>
    <td class="label">UserName:</td>
    <td><input name="UserName" type="text" size="40" value="AccountManagement" /></td>
  </tr>
  <tr>
    <td>Password:</td>
    <td><input name="Password" type="password" size="40" value="BlueSky27#"/></td>
  </tr>
  <tr>
    <td colspan="2"><hr/></td>
  </tr>
  <tr>
    <td>From:</td>
    <td><input name="SenderName" type="text" size="40" value="Tom Smith" /></td>
  </tr>
  <tr>
    <td>To:</td>
    <td>
      <input name="Recipient{" type="hidden" value="" />
      <input name="Name" type="text" size="40" value="Fred Meyer" />
    </td>
  </tr>
  <tr>
    <td>Fax Number:</td>
    <td>
      <input name="Address" type="text" size="40" value="503 597 5330" />
      <input name="}" type="hidden" value="" />
    </td>
  </tr>
  <tr>
    <td>Subject:</td>
    <td><input name="Subject" type="text" style="width:100%" value="This is a test" /></td>
  </tr>
  <tr>
    <td>CoverText:</td>
    <td>
      <input name="Document{" type="hidden" value="" />
      <input name="DocumentPart" type="hidden" value="0" />
      <textarea name="ContentText" style="width:100%;height:100px">This is a test.</textarea>
      <input name="}" type="hidden" value="" />
    </td>
  </tr>
  <tr>
    <td>Document:</td>
    <td>
      <input name="Document{" type="file" style="width:100%" value="" />
      <input name="DocumentPart" type="hidden" value="1" />
      <input name="}" type="hidden" value="" />
    </td>
  </tr>
</table>
<input name="ActionTypes" type="hidden" value="ClientOutbox" />
<input name="ResponseBody" type="hidden"
  value="<p>Status=$(ErrNum)<br/>StatusText=$(ErrStr)<br/>TemplateGuid=$(Id)</p>" />
<p><input type="submit" value="Submit" /></p>
</form>
</body>
</html>

```

Response 200 OK

Status=0
StatusText=Ok
TemplateGuid=S-7ac3e9e6-60a3-43ea-b9b5-3c1c9f176273

A network capture view of the posted data looks like this:

```
-----7e3376390fdc
Content-Disposition: form-data; name="UserName"

AccountManagement
-----7e3376390fdc
Content-Disposition: form-data; name="Password"

BlueSky27#
-----7e3376390fdc
Content-Disposition: form-data; name="SenderName"

Tom Smith
-----7e3376390fdc
Content-Disposition: form-data; name="Recipient{"

-----7e3376390fdc
Content-Disposition: form-data; name="Name"

Fred Meyer
-----7e3376390fdc
Content-Disposition: form-data; name="Address"

503 597 5330
-----7e3376390fdc
Content-Disposition: form-data; name="}"

-----7e3376390fdc
Content-Disposition: form-data; name="Subject"

This is a test
-----7e3376390fdc
Content-Disposition: form-data; name="Document{"

-----7e3376390fdc
Content-Disposition: form-data; name="DocumentPart"

0
-----7e3376390fdc
Content-Disposition: form-data; name="ContentText"

This is a test.
-----7e3376390fdc
Content-Disposition: form-data; name="}"

-----7e3376390fdc
Content-Disposition: form-data; name="Document{"; filename="Sample.tif"
Content-Type: image/tiff

II*
```

ReadQueue

Reads a comma-delimited list of message handles from one of the server's queues. A user can get a list of messages that belong to them, a supervisor can get a list of messages that belong to any user. The server can sort the list of message handles according to a specified column name.

Message handles are string values, that look similar to GUIDs.

To specify the source queue, include "Queue={queue-name}" in the URL. Queue names are: Send, Sending, Sent, Receiving, Received.

<https://api.nativefax.net/rest/Messages/ReadQueue?Queue=Sent&LoginId={MyLoginId}>

Parameter	Purpose												
Queue	The message queue to retrieve. The value can be the queue name or queue number: <table><tr><th>Queue Name</th><th>Queue Number</th></tr><tr><td>Send</td><td>0</td></tr><tr><td>Sending</td><td>1</td></tr><tr><td>Sent</td><td>2</td></tr><tr><td>Receiving</td><td>3</td></tr><tr><td>Received</td><td>4</td></tr></table>	Queue Name	Queue Number	Send	0	Sending	1	Sent	2	Receiving	3	Received	4
Queue Name	Queue Number												
Send	0												
Sending	1												
Sent	2												
Receiving	3												
Received	4												
OrderBy	Optional, one or more column names specifying the sort order for the queue query. Multiple column names should be separated with commas. Append "DESC" to the column name to request sorting a column in descending order, for example, "TotalSeconds DESC". For send queues, the specified column names can be in the FaxSend or FaxSendTransmission tables. AccountId, Status, StatusNum, Address, CallerID, LocalCSID, RemoteCSID, StartTime, Error, ErrorNumber, ExtendedErrorCode, DialSeconds, ConnectSeconds, TotalSeconds, ConnectBPS, Retries, Resolution, PageCount, PagesTransferred, SipCallId, ReceiverName, SenderName For receive queues, the specified column names can be in the FaxReceive table. AccountId, Address, CallerID, LocalCSID, RemoteCSID, StartTime, Error, ErrorNumber, ExtendedErrorCode, TotalSeconds, ConnectBPS, Resolution, PageCount Example (for a send queue): Address, TotalSeconds DESC												

AllUsers	Optional, if set to 1 then messages for all users in a queue are returned. The calling user must have supervisor privileges to use this option.						
AccountId -or- AccountGuid	Optional, if the calling user is a supervisor, then the queues for a specific user can be requested.						
Count	Optional, can be used to specify the maximum number of messages to return.						
ActionType	Optional, specifies an ActionType that a message must be associated with in order to be included in the response. If not specified, then returned messages are those that are associated with the default action type for the specified Queue. <table border="1"> <thead> <tr> <th>Queue</th><th>Default Action Type</th></tr> </thead> <tbody> <tr> <td>Send, Sending, Sent</td><td>ClientOutbox</td></tr> <tr> <td>Receiving, Received</td><td>ClientInbox</td></tr> </tbody> </table>	Queue	Default Action Type	Send, Sending, Sent	ClientOutbox	Receiving, Received	ClientInbox
Queue	Default Action Type						
Send, Sending, Sent	ClientOutbox						
Receiving, Received	ClientInbox						

Try it: [ReadQueue](#)

Example that finds two messages in the Sent queue:

Request(POST): https://api.nativefax.net/rest/Messages/ReadQueue?LoginId={MyLoginId}
<pre><NSX> <Queue> <AllUsers>1</AllUsers> </Queue> </NSX></pre>
Response 400 Bad Request
Response 401 Unauthorized
Response 200 OK
<pre><MessageHandles>S-6c078a3e-2b56-436a-8d24-a9844c029326,S-c569076b-3dd1-437b-98a2-fbdc32f7e5b3</MessageHandles></pre>

Example when there are no messages in a queue:

Request (GET)
Response 200 OK
<pre><MessageHandles/></pre>

Implementation:

The following demonstrates how NSX maps a request to read a queue from the dbFax fax engine.

dbFax FaxReceive Targets (receive side)

Receiving
PostFaxAction
ActionRequired
PendingDeletion

Queues (how they map to targets):

Queue Name	dbFax Routing Target
Receiving	Receiving
Received	PostFaxAction or ActionRequired
ReceivePendingDeletion	PendingDeletion

dbFax FaxSend Targets (send side)

New
Convert
FaxSend
InTransmission
PostFaxAction
ActionRequired
PendingDeletion

Queues (how they map to targets):

Queue Name	dbFax Routing Target
Send	New or Convert or FaxSend
Sending	InTransmission
Sent	PostFaxAction or ActionRequired
SendPendingDeletion	PendingDeletion

ReadMessageBlock

Reads a collection of messages from a list of provided message handles that were obtained by calling ReadQueue.

Try it: [ReadMessageBlock](#)

Parameter	Meaning
-----------	---------

ActionType	Optional, specifies the action type that a message must be associated with in order to be found. Important: Provide the same ActionType to ReadMessageBlock that was used when ReadQueue was called.
------------	--

Example ReadMessageBlock for sent messages:

Request(POST): https://api.nativefax.net/rest/Messages/ReadMessageBlock?LoginId={MyLoginId}
<pre><NSX> <MessageHandles> S-22b73931-af04-490a-aeb7-803b49dfe5a2,S-7c9fc59c-06aa-4003-880a-039bb19dc5ba </MessageHandles> </NSX></pre>
Response 400 Bad Request
Response 401 Unauthorized
Response 200 OK
<pre><NSX> <Message> <MessageHandle>S-22b73931-af04-490a-aeb7-803b49dfe5a2</MessageHandle> <Status>Ok</Status> <StatusNum>0</StatusNum> <Queue>2</Queue> <ActionType>ClientOutbox</ActionType> <AccountId>Supervisor</AccountId> <AccountGuid>ad56f7f2-4a4a-4649-a083-ae2b564c1aa7</AccountGuid> <Subject>Test</Subject> <Schedule>2017-07-09 17:57:35Z</Schedule> <SubmitTime>2017-07-09 17:57:35Z</SubmitTime> <SenderName/> <SenderFaxNumber/> <SenderVoiceNumber/> <SenderCompany/> <AccountBillingCode/> <MessageBillingCode/> <CoverTemplate/> <AppInfo/> <Resolution>516</Resolution> <Recipient> <RecipientGuid>e4b0c448-9efb-4d59-b2d1-f64109e03cdf</RecipientGuid> <Name>Fred</Name> <Address>5551234</Address> <OriginalAddress/> <Prefix>0</Prefix> <CallerId/> <SkipDestination>0</SkipDestination> <StartTime>2017-07-09 17:57:35Z</StartTime> <DialSeconds>1</DialSeconds> <ConnectSeconds>37</ConnectSeconds> <TotalSeconds>38</TotalSeconds> <PageCount>2</PageCount> <PagesTransferred>2</PagesTransferred> <ConnectBPS>14400</ConnectBPS> <Resolution>512</Resolution> <LocalCSId/> <RemoteCSId/> </Recipient> </Message> </NSX></pre>

```

<Retries>0</Retries>
<Error/>
<ErrorNumber>0</ErrorNumber>
<ExtendedErrorNumber>0</ExtendedErrorNumber>
<SendErrorType>0</SendErrorType>
<DialPrefixUsed/>
<SIPCallId>64018fce-db54-4697-ac10-0ad23fab7dc8@127.0.0.1</SIPCallId>
<RTNCount>0</RTNCount>
<RTPCount>0</RTPCount>
<ECMFrameErrorCount>0</ECMFrameErrorCount>
<V21FrameErrorCount>0</V21FrameErrorCount>
<NSF1/>
<NSF2/>
<DIS>A074xICQgYCYIAAAAAA</DIS>
<PortUsed>NET_PORT_0001</PortUsed>
</Recipient>
<Document>
  <DocumentGuid>b6c80daf-4d54-40da-bc55-0e444d07f7e9</DocumentGuid>
  <DocumentPart>0</DocumentPart>
  <Name>Content.txt</Name>
  <DocumentType>5</DocumentType>
  <PageCount>0</PageCount>
</Document>
<Document>
  <DocumentGuid>6c40cc43-fe8c-4072-9689-467274471e3e</DocumentGuid>
  <DocumentPart>1</DocumentPart>
  <Name>empty file.txt</Name>
  <DocumentType>5</DocumentType>
  <PageCount>0</PageCount>
</Document>
<RoutingHistory>Created[2017-07-09 17:57:35Z|NSX on NICKQAWIN10PRO|Ok],FaxSend[2017-07-09
17:57:49Z|Conversion Server on NICKQAWIN10PRO|All documents ready],InTransmission[2017-07-09
17:57:50Z|Transmission Server on NICKQAWIN10PRO|Fax to 5551234 queued for
transmission],PostFaxAction[2017-07-09 17:58:34Z|Transmission Server on NICKQAWIN10PRO|Transmissions
Complete]</RoutingHistory>
</Message>
</NSX>

```

Example ReadMessageBlock for received messages:

Request(POST):

```
<NSX>
  <MessageHandles>
    R-e1e0d4e9-3ded-4c08-820f-7821928626fc,R-22759ed4-990c-4b68-b0e0-afd8cf3d1c78
  </MessageHandles>
</NSX>
```

Response 400 Bad Request

Response 401 Unauthorized

Response 200 OK

```
<NSX>
  <Message>
    <MessageHandle>R-e1e0d4e9-3ded-4c08-820f-7821928626fc</MessageHandle>
    <Status>Ok</Status>
    <StatusNum>0</StatusNum>
    <Queue>4</Queue>
    <ActionType>ClientInbox</ActionType>
    <AccountId>Postmaster</AccountId>
    <AccountGuid>289c111e-a277-453e-923a-27e9edff79c7</AccountGuid>
    <Subject/>
    <Schedule>2017-07-09 18:25:41Z</Schedule>
    <SubmitTime>2017-07-09 18:25:41Z</SubmitTime>
    <SenderName/>
    <Resolution>512</Resolution>
    <Recipient>
      <RecipientGuid>e1e0d4e9-3ded-4c08-820f-7821928626fc</RecipientGuid>
      <Name/>
      <Address>5553451322</Address>
      <OriginalAddress/>
      <Prefix>0</Prefix>
      <CallerId/>
      <SkipDestination>0</SkipDestination>
      <StartTime>2017-07-09 18:25:41Z</StartTime>
      <DialSeconds>0</DialSeconds>
      <ConnectSeconds>38</ConnectSeconds>
      <TotalSeconds>38</TotalSeconds>
      <PageCount>2</PageCount>
      <PagesTransferred>2</PagesTransferred>
      <ConnectBPS>14400</ConnectBPS>
      <Resolution>512</Resolution>
      <LocalCSId/>
      <RemoteCSId/>
      <Retries>0</Retries>
      <Error/>
      <ErrorNumber>0</ErrorNumber>
      <ExtendedErrorNumber>0</ExtendedErrorNumber>
      <SendErrorType>0</SendErrorType>
      <DialPrefixUsed/>
      <SIPCallId>7d075abd-a583-403c-9967-
ee583745f2dd@127.0.0.1<!--Postmaster-->postmaster<!--b31d4d1936b44c5bbb494bcb57614791-->
<!--289c111e-a277-453e-923a-27e9edff79c7-->Topmost Domain<!--b2984b93-b6cf-4025-9b8e-
c661eabfcbad</SIPCallId>
      <RTNCount>0</RTNCount>
      <RTPCount>0</RTPCount>
      <ECMFrameErrorCount>0</ECMFrameErrorCount>
      <V21FrameErrorCount>0</V21FrameErrorCount>
      <NSF1/>
      <NSF2/>
      <DIS/>
```



```

    <PortUsed>NET_PORT_0004</PortUsed>
  </Recipient>
  <Document>
    <DocumentGuid>e1e0d4e9-3ded-4c08-820f-7821928626fc</DocumentGuid>
    <DocumentPart>1</DocumentPart>
    <Name/>
    <DocumentType>1</DocumentType>
    <PageCount>2</PageCount>
  </Document>
  <RoutingHistory>Receiving[2017-07-09 18:25:41Z|Transmission Server on NICKQAWIN10PRO|New incoming
fax],PostFaxAction[2017-07-09 18:26:21Z|Transmission Server on NICKQAWIN10PRO|Completed receive for
170709-182541304-2FD50004]</RoutingHistory>
</Message>
</NSX>

```

BuildFaxImage

Returns the entire content of a sent or received fax message in a single image file. Depending on how the request is made, the fax image is either returned encoded in base64 in an Xml/Json response or the fax image is returned as the sole content.

Try it: [BuildFaxImage](#)

Example of BuildFaxImage using HTTP GET

When HTTP GET is used to call BuildFaxImage, all parameters must be included in the URL. For credentials, either include an Authorization header or in a separate call login to the server with the Login function and obtain a LoginId and then include the LoginId in the URL to call BuildFaxImage.

For a successful response, the fax image will be returned.

<https://api.nativefax.net/rest/Messages/BuildFaxImage?MessageHandle=R-ece3100a-b996-4b15-9915-3394f03dbd60&LoginId=058cd911-813d-4a94-b2b9-55e877b0e23f>

Request(GET):

Authorization: Basic c3VwZXJ2aXNvcjo=

Response 400 Bad Request

Response 401 Unauthorized

Response 200 OK

```

200 OK
Content-Disposition: attachment; filename="Fax.tif"
Content-Length: 1824
Content-Type: image/tiff
Date: Mon, 10 Jul 2017 23:32:15 GMT
Server: NSApplications 9.2.8910.640

II*...(THE REST OF THE FILE)

```

The following example shows how to request the image in PDF format, by including **DocumentType=PDF** in the query:

Request(GET):

<http://<host>/mqs/Messages/BuildFaxImage?DocumentType=PDF&MessageHandle=R-ece3100a-b996-4b15-9915-3394f03dbd60>

Authorization: Basic c3VwZXJ2aXNvcjo=

Response 400 Bad Request**Response 401 Unauthorized****Response 200 OK**

200 OK
 Content-Disposition: attachment; filename="Fax.pdf"
 Content-Length: 3824
 Content-Type: application/pdf
 Date: Mon, 10 Jan 2018 23:32:15 GMT
 Server: NSApplications 9.2.8910.640
 %PDF-1.1...(THE REST OF THE PDF FILE)

Example of BuildFaxImage using HTTP POST

When HTTP POST is used to call BuildFaxImage, the fax image is returned in a ContentData node encoded in base64 as part of the Xml/Json response.

Request (POST): <https://api.nativefax.net/rest/Messages/BuildFaxImage?LoginId={MyLoginId}>

```
<HttpService>
  <NSX>
    <BuildFaxImage>
      <MessageHandle>R-ece3100a-b996-4b15-9915-3394f03dbd60</MessageHandle>
    </BuildFaxImage>
  </NSX>
</HttpService>
```

Response 400 Bad Request**Response 401 Unauthorized****Response 200 OK**

```
<HttpService version='1.0'>
  <NSXResponse>
    <BuildFaxImageResponse>
      <DocumentType>1</DocumentType>
      <Resolution>512</Resolution>
      <PageCount>1</PageCount>

      <ContentData>SUKqAAgAAAASAP4ABAABAAAAAgAAAAABBAABAAAAwAYAAAEBBABAAAAjwgAAAIbAwABAAAAAQAAAAMBawABAAAAABAAAA
      AYBAwABAAAAAaAwABAAAAAgAAABEBBAABAAAAANwEAABIBAwABAAAAAQAAABUBAwABAAAAAQAAABYBBABAAAAjwgAABcBBABAAAA
      A6QUAABoBBQABAAAA5gAAABsBBQABAAAA7gAAACgBAwABAAAAAgAAACKBAwACAAAAAABADEBAgAbAAAA9gAAAJiCagAmAAAAEQEAAAAAA
      ADMAAAAAQAAAMQAAAAABAAAAARmF4QmFjayBwb0lQIEZheCBTb2x1dG1vbnMAQ29weXJpZ2h0ICVhYmMDEzLTlwMTcgRmF4...(THE REST
      OF THE FILE)</ContentData>
      <Status>Ok</Status>
      <StatusNum>0</StatusNum>
    </BuildFaxImageResponse>
    <Status>Ok</Status>
    <StatusNum>0</StatusNum>
  </NSXResponse>
  <StatusNum>0</StatusNum>
```

```
<Status>Command Complete</Status>  
</HttpService>
```

ReadMessage

Reads a single message

Try it: [ReadMessage](#)

Example ReadMessage for a sent fax:

Request(POST): <https://api.nativefax.net/rest/Messages/ReadMessage?LoginId={MyLoginId}>

```
<NSX>
  <Message>
    <MessageHandle>S-7c9fc59c-06aa-4003-880a-039bb19dc5ba</MessageHandle>
  </Message>
</NSX>
```

Response 400 Bad Request

Response 401 Unauthorized

Response 200 OK

```
<Message>
  <MessageHandle>S-7c9fc59c-06aa-4003-880a-039bb19dc5ba</MessageHandle>
  <Status>0</Status>
  <StatusNum>0</StatusNum>
  <Queue>2</Queue>
  <ActionType>ClientOutbox</ActionType>
  <AccountId>Supervisor</AccountId>
  <AccountGuid>ad56f7f2-4a4a-4649-a083-ae2b564c1aa7</AccountGuid>
  <Subject>Test</Subject>
  <Schedule>2017-07-09 18:25:38Z</Schedule>
  <SubmitTime>2017-07-09 18:25:38Z</SubmitTime>
  <SenderName/>
  <SenderFaxNumber/>
  <SenderVoiceNumber/>
  <SenderCompany/>
  <AccountBillingCode/>
  <MessageBillingCode/>
  <CoverTemplate/>
  <AppInfo/>
  <Resolution>516</Resolution>
  <Recipient>
    <RecipientGuid>1b508231-7109-41dd-a832-dc3383f32667</RecipientGuid>
    <Name>Fred</Name>
    <Address>5553451322</Address>
    <OriginalAddress/>
    <Prefix>0</Prefix>
    <CallerId/>
    <SkipDestination>0</SkipDestination>
    <StartTime>2017-07-09 18:25:38Z</StartTime>
    <DialSeconds>0</DialSeconds>
    <ConnectSeconds>37</ConnectSeconds>
    <TotalSeconds>37</TotalSeconds>
    <PageCount>2</PageCount>
    <PagesTransferred>2</PagesTransferred>
    <ConnectBPS>14400</ConnectBPS>
    <Resolution>512</Resolution>
    <LocalCSId/>
    <RemoteCSId/>
    <Retries>0</Retries>
    <Error/>
    <ErrorNumber>0</ErrorNumber>
    <ExtendedErrorNumber>0</ExtendedErrorNumber>
    <SendErrorType>0</SendErrorType>
    <DialPrefixUsed/>
    <SIPCallId>7d075abd-a583-403c-9967-ee583745f2dd@127.0.0.1</SIPCallId>
    <RTNCount>0</RTNCount>
    <RTPCount>0</RTPCount>
    <ECMFrameErrorCount>0</ECMFrameErrorCount>
```

```

<V21FrameErrorCount>0</V21FrameErrorCount>
<NSF1/>
<NSF2/>
<DIS>A074xICQgYCYIAAAAAA</DIS>
<PortUsed>NET_PORT_0003</PortUsed>
</Recipient>
<Document>
  <DocumentGuid>46cad650-c5af-4bbc-900c-1840ae8c3285</DocumentGuid>
  <DocumentPart>0</DocumentPart>
  <Name>Content.txt</Name>
  <DocumentType>5</DocumentType>
  <PageCount>0</PageCount>
</Document>
<Document>
  <DocumentGuid>1b5a2d49-fd55-41fe-8f57-d0a7f0a1eeae</DocumentGuid>
  <DocumentPart>1</DocumentPart>
  <Name>empty file.txt</Name>
  <DocumentType>5</DocumentType>
  <PageCount>0</PageCount>
</Document>
<RoutingHistory>Created[2017-07-09 18:25:38Z|NSX on NICKQAWIN10PRO|Ok],FaxSend[2017-07-09
18:25:38Z|Conversion Server on NICKQAWIN10PRO|All documents ready],InTransmission[2017-07-09
18:25:40Z|Transmission Server on NICKQAWIN10PRO|Fax to 5553451322 queued for
transmission],PostFaxAction[2017-07-09 18:26:21Z|Transmission Server on NICKQAWIN10PRO|Transmissions
Complete]</RoutingHistory>
</Message>

```

Example ReadMessage for a received fax that was received successfully:

Request(POST): <https://api.nativefax.net/rest/Messages/ReadMessage?LoginId={MyLoginId}>

```

<NSX>
  <Message>
    <MessageHandle>R-e1e0d4e9-3ded-4c08-820f-7821928626fc</MessageHandle>
  </Message>
</NSX>

```

Response 400 Bad Request

Response 401 Unauthorized

Response 200 OK

```

<Message>
  <MessageHandle>R-e1e0d4e9-3ded-4c08-820f-7821928626fc</MessageHandle>
  <Status>0k</Status>
  <StatusNum>0</StatusNum>
  <Queue>4</Queue>
  <ActionType>ClientInbox</ActionType>
  <AccountId>Postmaster</AccountId>
  <AccountGuid>289c111e-a277-453e-923a-27e9edff79c7</AccountGuid>
  <Subject/>
  <Schedule>2017-07-09 18:25:41Z</Schedule>
  <SubmitTime>2017-07-09 18:25:41Z</SubmitTime>
  <SenderName/>
  <Resolution>512</Resolution>
  <Recipient>
    <RecipientGuid>e1e0d4e9-3ded-4c08-820f-7821928626fc</RecipientGuid>
    <Name/>
    <Address>5553451322</Address>
    <OriginalAddress/>
    <Prefix>0</Prefix>
    <CallerId/>
  </Recipient>

```

```

<SkipDestination>0</SkipDestination>
<StartTime>2017-07-09 18:25:41Z</StartTime>
<DialSeconds>0</DialSeconds>
<ConnectSeconds>38</ConnectSeconds>
<TotalSeconds>38</TotalSeconds>
<PageCount>2</PageCount>
<PagesTransferred>2</PagesTransferred>
<ConnectBPS>14400</ConnectBPS>
<Resolution>512</Resolution>
<LocalCSId/>
<RemoteCSId/>
<Retries>0</Retries>
<Error/>
<ErrorNumber>0</ErrorNumber>
<ExtendedErrorNumber>0</ExtendedErrorNumber>
<SendErrorType>0</SendErrorType>
<DialPrefixUsed/>
<SIPCallId>7d075abd-a583-403c-9967-
ee583745f2dd@127.0.0.1<&lt;&gt;Postmaster&lt;&gt;postmaster&lt;&gt;b31d4d1936b44c5bbb494bcb57614791&lt;&gt;
&lt;&gt;289c111e-a277-453e-923a-27e9edff79c7&lt;&gt;Topmost Domain&lt;&gt;b2984b93-b6cf-4025-9b8e-
c661eabfcbad</SIPCallId>
<RTNCount>0</RTNCount>
<RTPCount>0</RTPCount>
<ECMFrameErrorCount>0</ECMFrameErrorCount>
<V21FrameErrorCount>0</V21FrameErrorCount>
<NSF1/>
<NSF2/>
<DIS/>
<PortUsed>NET_PORT_0004</PortUsed>
</Recipient>
<Document>
<DocumentGuid>e1e0d4e9-3ded-4c08-820f-7821928626fc</DocumentGuid>
<DocumentPart>1</DocumentPart>
<Name/>
<DocumentType>1</DocumentType>
<PageCount>2</PageCount>
</Document>
<RoutingHistory>Receiving[2017-07-09 18:25:41Z|Transmission Server on TEST1|New incoming
fax],PostFaxAction[2017-07-09 18:26:21Z|Transmission Server on TEST1|Completed receive for 170709-
182541304-2FD50004]</RoutingHistory>
</Message>

```

Example ReadMessage for a received fax that was received with an error. For a received message, the message-level status (Status and StatusNum) are identical to the Error and ErrorNumber in the one-and-only Recipient.

Request(POST): <https://api.nativefax.net/rest/Messages/ReadMessage?LoginId={MyLoginId}>

```

<NSX>
  <Message>
    <MessageHandle>R-21269eb2-9ec7-4113-b36b-69665d00d7ef</MessageHandle>
  </Message>
</NSX>

```

Response 400 Bad Request

Response 401 Unauthorized

Response 200 OK

```

<Message>
  <MessageHandle>R-21269eb2-9ec7-4113-b36b-69665d00d7ef</MessageHandle>

```

```

<Status>Call was hung up (received BYE)</Status>
<StatusNum>30243</StatusNum>
<Queue>4</Queue>
<ActionType>ClientInbox</ActionType>
<AccountId>Postmaster</AccountId>
<AccountGuid>289c111e-a277-453e-923a-27e9edff79c7</AccountGuid>
<Subject/>
<Schedule>2017-07-09 18:25:41Z</Schedule>
<SubmitTime>2017-07-09 18:25:41Z</SubmitTime>
<SenderName/>
<Resolution>512</Resolution>
<Recipient>
  <RecipientGuid>21269eb2-9ec7-4113-b36b-69665d00d7ef</RecipientGuid>
  <Name/>
  <Address>5553451322</Address>
  <OriginalAddress/>
  <Prefix>0</Prefix>
  <CallerId>1234</CallerId>
  <SkipDestination>0</SkipDestination>
  <StartTime>2017-07-09 18:25:41Z</StartTime>
  <DialSeconds>0</DialSeconds>
  <ConnectSeconds>138</ConnectSeconds>
  <TotalSeconds>138</TotalSeconds>
  <PageCount>2</PageCount>
  <PagesTransferred>2</PagesTransferred>
  <ConnectBPS>28800</ConnectBPS>
  <Resolution>512</Resolution>
  <LocalCSId>1234</LocalCSId>
  <RemoteCSId>1234</RemoteCSId>
  <Retries>0</Retries>
  <Error>Call was hung up (received BYE)</Error>
  <ErrorNumber>30243</ErrorNumber>
  <ExtendedErrorNumber>0</ExtendedErrorNumber>
  <SendErrorType>0</SendErrorType>
  <DialPrefixUsed/>
  <SIPCallId>7d075abd-a583-403c-9967-
ee583745f2dd@127.0.0.1<!-->Postmaster<!-->postmaster<!-->b31d4d1936b44c5bbb494bcb57614791<!-->
<!-->289c111e-a277-453e-923a-27e9edff79c7<!-->Topmost Domain<!-->b2984b93-b6cf-4025-9b8e-
c661eabfcbad</SIPCallId>
  <RTNCount>0</RTNCount>
  <RTPCount>0</RTPCount>
  <ECMFrameErrorCount>0</ECMFrameErrorCount>
  <V21FrameErrorCount>0</V21FrameErrorCount>
  <NSF1/>
  <NSF2/>
  <DIS/>
  <PortUsed>NET_PORT_0004</PortUsed>
</Recipient>
<Document>
  <DocumentGuid>21269eb2-9ec7-4113-b36b-69665d00d7ef</DocumentGuid>
  <DocumentPart>1</DocumentPart>
  <Name/>
  <DocumentType>1</DocumentType>
  <PageCount>2</PageCount>
</Document>
<RoutingHistory>Receiving[2017-07-09 18:25:41Z|Transmission Server on QAWIN10PRO|New incoming
fax],PostFaxAction[2017-07-09 18:26:21Z|Transmission Server on QAWIN10PRO|Completed receive for 170709-
182541304-2FD50004],Copied[2017-07-09 18:29:47Z|NSX on QAWIN10PRO|Action(s): ClientInbox, source handle:
R-e1e0d4e9-3ded-4c08-820f-7821928626fc]</RoutingHistory>
</Message>

```

Components of a Message

Parameter	Meaning
Document/DocumentType	Unknown = 0, TIFF = 1, RTF = 2, PDF = 3, HTML = 4, TEXT = 5,
Resolution	HUFFMAN = 1, TWO_D_HUFFMAN = 2, T6 = 4, JPEG = 16, COLOR = 32, DPI_200H_100V = 256, DPI_200H_200V = 512, DPI_200H_400V = 1024, DPI_300H_300V = 2048, DPI_400H_400V = 4096, WIDTH_255MM = 8192, WIDTH_303MM = 16384

DeleteMessage

Deletes one or more messages. Multiple messages may be deleted by providing a comma-delimited list of message handles.

Try it: [DeleteMessage](#)

Request(POST): https://api.nativefax.net/rest/Messages/DeleteMessage?LoginId={MyLoginId}
<pre><NSX> <Message> <MessageHandle>S-9b4b6e2e-6352-4ea6-aedd-b6215cd889e7</MessageHandle> </Message> </NSX></pre>
Response 400 Bad Request
Response 401 Unauthorized
Response 200 OK

AbortMessage

Aborts messages in the Sending or Receiving queue.

Try it: [AbortMessage](#)

Request(POST): https://api.nativefax.net/rest/Messages/AbortMessage?LoginId={MyLoginId}

```
<NSX>
  <Message>
    <MessageHandle>S-9b4b6e2e-6352-4ea6-aedd-b6215cd889e7</MessageHandle>
  </Message>
</NSX>
```

Response 400 Bad Request

Response 401 Unauthorized

Response 200 OK

GetQueueCounts

Returns counts for all or one (if specified) account(s) in a queue.

Parameter	Meaning
AccountId -or- AccountGuid	Optional, specifies an account where only counts for that account will be returned. If not specified, then counts are returned for all accounts in a queue.
Action Type	Optional, specifies an action type (e.g., AutoPrint, ATA Delivery, etc.) and only counts related to that action type will be returned.
AllUsers	Optional, if set to 1 then message count for all users in a queue are returned. The calling user must have supervisor privileges to use this option.

Try it: [GetQueueCounts](#)

Request(GET): <https://api.nativefax.net/rest/Messages/GetQueueCounts?LoginId={MyLoginId}>

Response 400 Bad Request

Response 401 Unauthorized

Response 200 OK

```
<QueueCounts>
  <Send>1</Send>
  <Sending>0</Sending>
  <Sent>3</Sent>
  <Receiving>0</Receiving>
  <Received>0</Received>
  <FaxEngine>Default</FaxEngine>
</QueueCounts>
```

API Object References

Note: I [Quentin] added the account created screens. I'm working on updating the label in the table to match better so it's easier to see what's what.

When using the NSX API you will need to supply data in the post with what properties you are creating your accounts or managing Groups & DID's. Below is a reference list of the properties that can be used in the different variations of API calls to the NSX Service.

Account Object References

Property	Description	ClientType	Json Example
AccountId	Name of the account	All Types	"AccountId": "MyAccount",
PasswordStr	The account password	All Types	"PasswordStr": "B1ueSky27#",
ClientType	The Account Type	All Types	"ClientType": "0", Bit Values: 0 = None 1 = FaxATA 2 = Email 4 = FAXability 8 = MSFax/API 16 = Realtime ** A ClientType of "7", would FaxATA, Email, & FAXability ** Realtime cannot be combined with other Client Types and must be set by itself.
AllowSnd	Enable/Disable Send	All Types	"AllowSnd": "1", "AllowSnd": "0",
AllowRcv	Enable/Disable Receive	All Types	"AllowRcv": "1", "AllowRcv": "0",

Account

Profile

Delivery & Notif.

Inbound Routing

Call Identity

Advanced

Covers

Groups

Account Profile Settings

Prefix

First Name

Middle Name

Last Name

Suffix

Email

Phone

Cell

Company

Description

Language

Time Zone

Quentin

J.

Dible

quentin@faxback.com

FaxBack

NSX API Account

English

Pacific Standard Time

Save

Cancel

Property	Description	ClientType	Json Example
Prefix	Sets the Prefix in the Account Profile	All Types	"Prefix": "Mr.",
First	Sets the First Name in the Account Profile	All Types	"First": "John",
Middle	Sets the Middle Name in the Account Profile	All Types	"Middle": "J.",
Last	Sets the Last Name in the Account Profile	All Types	"Last": "Johnson",
Suffix	Sets the Suffix in the Account Profile	All Types	"Suffix": "Sr.",
Address1	Sets Address1 in the Account Profile	All Types	"Address1": "7007 SW Cardinal Lane",
Address2	Sets Address2 in the Account Profile	All Types	"Address2": "Suite 105",
City	Sets the City in the Account Profile	All Types	"City": "Portland",
State	Sets the State in the Account Profile	All Types	"State": "Oregon",
PostalCode	Sets the Postal Code in the Account Profile	All Types	"PostalCode": "97224",
Country	Sets the Country in the Account Profile	All Types	"Country": "United States",
Company	Sets the Company in the Account Profile	All Types	"Company": "Faxback",
ContactEmail	Sets the Email in the Account Profile. This also is used for Password Reset.	All Types	"ContactEmail": "NotARealEmail@faxback.com",
OfficePhone	Sets the Office Phone in the Account Profile	All Types	"OfficePhone": "5035551212",
CellPhone	Sets the Cell Phone in the Account Profile	All Types	"CellPhone": "5035551212",
Description	Account Description	All Types	"Description": "Office",
QueueProfileXml	Sets many values pertaining to delivery notifications.	All Types	Tz = TimeZone Ln = Sets the language

Account

Profile

Delivery & Notif.

Inbound Routing

Call Identity

Advanced

Covers

Groups

Call Identity Settings

ANI

3605551042

CSID

3605551042

TSID

3605551042

Billing Code

Quentin's NSX API Account

☒ Create CDR's for this account

Save

Cancel

Property	Description	ClientType	Json Example
BillingCode	Set the BillingCode for the Account.	All Types	"BillingCode": "A1B2C3",
CSID	Sets the CSID at the account level	All Types	"CSID": "5415381234",
TSID	Sets the TSID at the account level	All Types	"TSID": "5415381234",
ANI	Sets the ANI at the account level	All Types	"ANI": "5415381234",
SaveCDR	Enable Saving CDRs for this account. This is on by default.	All Types	"SaveCDR": "1", "SaveCDR": "0",

QueueProfileXml Reference (full description and details)

QueueProfileXml Property	Description	ClientType	Json Example
Rt = Receive type	Where to route faxes and notifications to. These can be added together to select more than one receive type.	All Types	Rt = 1 = ATA 2 = Email 4 = FAXability 8 = Client inbox
Ea = Email Address for delivery notification			Ea = Email address for Delivery Notifications and Received Faxes
Ef = Attachment format			Ef = Email Attachment format. 0=tif 1=pdf
Dn = Delivery notifications			Dn = Delivery notifications 0 = None 1 = ATA w/thumbnail 1/8 = ATA with no thumbnail 2 = Email Notification On 2/1 = Email w/faxed image 2/2 = Email w/Orig Attachments 2/3 = Email w/both original and faxed images attached. For combinations, add the values together. Example: Printed w/thumbnail = 1 Email with both attachment types = 2/3 Resulting value = 3/3
Ndn = Non-delivery notifications			Ndn = Non-delivery notifications. Same values apply as Dn above.
Ntfstart = FAXability			Ntfstart = Enable or disable delivery notifications when sending from the FAXability Client. 1 = Enabled 0 = Disabled

To do: The following have **not yet** been matched up to the Administration Account Management Web GUI.

Customer	Customer for reference	ATA Account	"Customer": "Medical Company",
Source	Source for reference	ATA Account	"Source": "Verizon",
CallAhead	This enabled the Call Ahead feature on a Fax ATA	ATA Account	"CallAhead": "1", "CallAhead": "0",
ClientLogs	Enable sending client logs to the service.	ATA Account	"ClientLogs": "1", "ClientLogs": "0",
LoginGroup	Sets the Fax ATA login group	ATA Account	"LoginGroup": "GroupA",
RuleGroup	Sets the Fax ATA send rule group	ATA Account	"RuleGroup": "SendRulesA",
AllowSndOnBehalfOf	Enabled Send on Behalf. Primarily used by the ATA QP Account.	Realtime	"AllowSndOnBehalfOf": "0", "AllowSndOnBehalfOf": "1",
RcvFailoverGuid	Sets the failover account for receiving.	FaxATA	"RcvFailoverGuid": "00000000-0000-0000-0000-000000000000",
RcvFailoverMode	Set the mode in which the Fax ATA will receive.	FaxATA	Always Receive "RcvFailoverMode": "1", Only when logged in. "RcvFailoverMode": "5",
FaxATAType	Set the type or model of FaxATA	FaxATA	"FaxATAType": "-1", Unknown "FaxATAType": "0", MP202B "FaxATAType": "1", MP202D "FaxATAType": "2", MP264B "FaxATAType": "3", LEXMARK "FaxATAType": "4", FV_G201N4 "FaxATAType": "5", MP202R "FaxATAType": "6", RN_QX300
ProvisioningArg ***	Sets the Provisioning Argument	Fax ATA	"ProvisioningArg": "",
SerialNumber	Sets a SerialNumber to the FaxATA Account	Fax ATA	"SerialNumber": "D01234567",
MACAddress	Sets a SerialNumber to the FaxATA Account	Fax ATA	"MACAddress": "00908F12D687",
EmailCoverType		Email Account	"EmailCoverType": "1",
UseCoverPage	Enable use of a Cover Page	Email Account	"UseCoverPage": "1", "UseCoverPage": "0",
CoverPage	The name of the cover page to be used	Email Account	"CoverPage": "MyCoverPage.rtf",
PIN ***	Sets the PIN for an Account	Fax ATA	"PIN": "",
PINMode ***	Sets the Pin Mode on the Account	Fax ATA	"PINMode": "0",
PINGuid ***	Sets the PINGuid on the Account	Fax ATA	"PINGuid": "00000000-0000-0000-0000-000000000000",
PW	This is where account Password for Auto Provisioning is encoded and stored temporarily	Fax ATA	
PWExpires	Expiration datetime for the PW value for auto provisioning	Fax ATA	
SndTransport ***	Sets the SndTransport field	All Types	"SndTransport": "",
RawXml / AdminNotes	Sets the Admin Notes for the account.	All Types	"RawXml": { "AdminNotes": "Notes Here" },
DomainGuid	Sets the DomainGuid for the account	All Types	"DomainGuid": "9789f6bb-0293-49a0-8262-adc8291c218a",
DomainId	Sets the DomainId for the account	All Types	"DomainId": "Topmost Domain",

Property	Description	ClientType	Json Example
AccountId	Name of the account	All Types	"AccountId": "MyAccount",
PasswordStr	The account password	All Types	"PasswordStr": "BlueSky27#",
ClientType	The Account Type	All Types	"ClientType": "0", Bit Values: 0 = None 1 = FaxATA 2 = Email 4 = FAXability 8 = MSFax/API 16 = Realtime ** A ClientType of "7", would FaxATA, Email, & FAXability ** Realtime cannot be combined with other Client Types and must be set by itself.
Description	Account Description	All Types	"Description": "Office",
Customer	Customer for reference	ATA Account	"Customer": "Medical Company",
Source	Source for reference	ATA Account	"Source": "Verizon",
CSID	Sets the CSID at the account level	All Types	"CSID": "5415381234",
TSID	Sets the TSID at the account level	All Types	"TSID": "5415381234",
ANI	Sets the ANI at the account level	All Types	"ANI": "5415381234",
SaveCDR	Enable Saving CDRs for this account. This is on by default.	All Types	"SaveCDR": "1", "SaveCDR": "0",
CallAhead	This enabled the Call Ahead feature on a Fax ATA	ATA Account	"CallAhead": "1", "CallAhead": "0",
ClientLogs	Enable sending client logs to the service.	ATA Account	"ClientLogs": "1", "ClientLogs": "0",
LoginGroup	Sets the Fax ATA login group	ATA Account	"LoginGroup": "GroupA",
RuleGroup	Sets the Fax ATA send rule group	ATA Account	"RuleGroup": "SendRulesA",
AllowSnd	Enable/Disable Send	All Types	"AllowSnd": "1", "AllowSnd": "0",
AllowRcv	Enable/Disable Receive	All Types	"AllowRcv": "1", "AllowRcv": "0",
AllowSndOnBehalfOf	Enabled Send on Behalf. Primarily used by the ATA QP Account.	Realtime	"AllowSndOnBehalfOf": "0", "AllowSndOnBehalfOf": "1",
RcvFailoverGuid	Sets the failover account for receiving.	FaxATA	"RcvFailoverGuid": "00000000-0000-0000-0000-000000000000",
RcvFailoverMode	Set the mode in which the Fax ATA will receive.	FaxATA	Always Receive "RcvFailoverMode": "1", Only when logged in. "RcvFailoverMode": "5",
FaxATAType	Set the type or model of FaxATA	FaxATA	"FaxATAType": "-1", Unknown "FaxATAType": "0", MP202B "FaxATAType": "1", MP202D "FaxATAType": "2", MP264B "FaxATAType": "3", LEXMARK "FaxATAType": "4", FV_G201N4 "FaxATAType": "5", MP202R "FaxATAType": "6", RN_QX300
ProvisioningArg ***	Sets the Provisioning Argument	Fax ATA	"ProvisioningArg": "",
SerialNumber	Sets a SerialNumber to the FaxATA Account	Fax ATA	"SerialNumber": "D01234567",
MACAddress	Sets a SerialNumber to the FaxATA Account	Fax ATA	"MACAddress": "00908F12D687",
EmailCoverType		Email Account	"EmailCoverType": "1",

UseCoverPage	Enable use of a Cover Page	Email Account	"UseCoverPage": "1", "UseCoverPage": "0",
CoverPage	The name of the cover page to be used	Email Account	"CoverPage": "MyCoverPage.rtf",
BillingCode	Set the BillingCode for the Account.	All Types	"BillingCode": "A1B2C3",
PIN ***	Sets the PIN for an Account	Fax ATA	"PIN": "",
PINMode ***	Sets the Pin Mode on the Account	Fax ATA	"PINMode": "0",
PINGuid ***	Sets the PINGuid on the Account	Fax ATA	"PINGuid": "00000000-0000-0000-0000-000000000000",
Prefix	Sets the Prefix in the Account Profile	All Types	"Prefix": "Mr.",
First	Sets the First Name in the Account Profile	All Types	"First": "John",
Middle	Sets the Middle Name in the Account Profile	All Types	"Middle": "J.",
Last	Sets the Last Name in the Account Profile	All Types	"Last": "Johnson",
Suffix	Sets the Suffix in the Account Profile	All Types	"Suffix": "Sr.",
Address1	Sets Address1 in the Account Profile	All Types	"Address1": "7007 SW Cardinal Lane",
Address2	Sets Address2 in the Account Profile	All Types	"Address2": "Suite 105",
City	Sets the City in the Account Profile	All Types	"City": "Portland",
State	Sets the State in the Account Profile	All Types	"State": "Oregon",
PostalCode	Sets the Postal Code in the Account Profile	All Types	"PostalCode": "97224",
Country	Sets the Country in the Account Profile	All Types	"Country": "United States",
Company	Sets the Company in the Account Profile	All Types	"Company": "Faxback",
ContactEmail	Sets the Email in the Account Profile. This also is used for Password Reset.	All Types	"ContactEmail": "NotARealEmail@faxback.com",
PW	This is where account Password for Auto Provisioning is encoded and stored temporarily	Fax ATA	
PWExpires	Expiration datetime for the PW value for auto provisioning	Fax ATA	
OfficePhone	Sets the Office Phone in the Account Profile	All Types	"OfficePhone": "5035551212",
CellPhone	Sets the Cell Phone in the Account Profile	All Types	"CellPhone": "5035551212",
SndTransport ***	Sets the SndTransport field	All Types	"SndTransport": "",

QueueProfileXml	Sets many values pertaining to delivery notifications.	All Types	Tz = TimeZone Rt = Receive type. Where to route faxes and notifications to. These can be added together to select more than one receive type. 1 = ATA 2 = Email 4 = FAXability 8 = Client inbox Ea = Email address for Delivery Notifications and Received Faxes Ef = Email Attachment format. 0=tif 1=pdf Dn = Delivery notifications 0 = None 1 = ATA w/thumbnail 1/8 = ATA with no thumbnail 2 = Email Notification On 2/1 = Email w/faxed image 2/2 = Email w/Orig Attachments 2/3 = Email w/both original and faxed images attached. For combinations, add the values together. Example: Printed w/thumbnail = 1 Email with both attachment types = 2/3 Resulting value = 3/3 Ndn = Non-delivery notifications. Same values apply as Dn above. Ntfstar = Enable or disable delivery notifications when sending from the FAXability Client. 1 = Enabled 0 = Disabled Ln = Sets the language <pre>"QPXml": { "@Tz": "Pacific*", "@Rt": "2", "@Ea": "NotARealEmail@faxback.com", "@Ef": "2", "@Dn": "2/1", "@Ndn": "2/2", "@Ntfstar": "1", "@Ln": "English" }</pre>
RawXml / AdminNotes	Sets the Admin Notes for the account.	All Types	<pre>"RawXml": { "AdminNotes": "Notes Here" },</pre>

DomainGuid	Sets the DomainGuid for the account	All Types	"DomainGuid": "9789f6bb-0293-49a0-8262-adc8291c218a",
DomainId	Sets the DomainId for the account	All Types	"DomainId": "Topmost Domain",

*** These are used specifically for Fax-to-Direct messaging from NSX.

DID Object References

Property	Description	Json Example
DID	Sets the DID property	"DID": "5411234567",
RcvAnswer	Enables or Disabled the DID	"RcvAnswer": "0",
AccountId		"AccountId": "FaxATA-D01234567",
Customer		"Customer": "CustomerName",
Source		"Source": "SourceName",
LineNum		"LineNum": "0", "LineNum": "1", "LineNum": "2",
LineId		"LineId": "5411234567",
CSID		"CSID": "5411234567",
PINMode ***	Sets the Pin Mode on the Account	"PIN": "",
PIN ***	Sets the PIN on the Account	"PINGuid": "00000000-0000-0000-0000-000000000000",
SndTransport ***	Sets the SndTransport field	"SndTransport": "",
RcvFwdToGuid		
QPXml		

*** These are used specifically for Fax-to-Direct messaging from NSX.

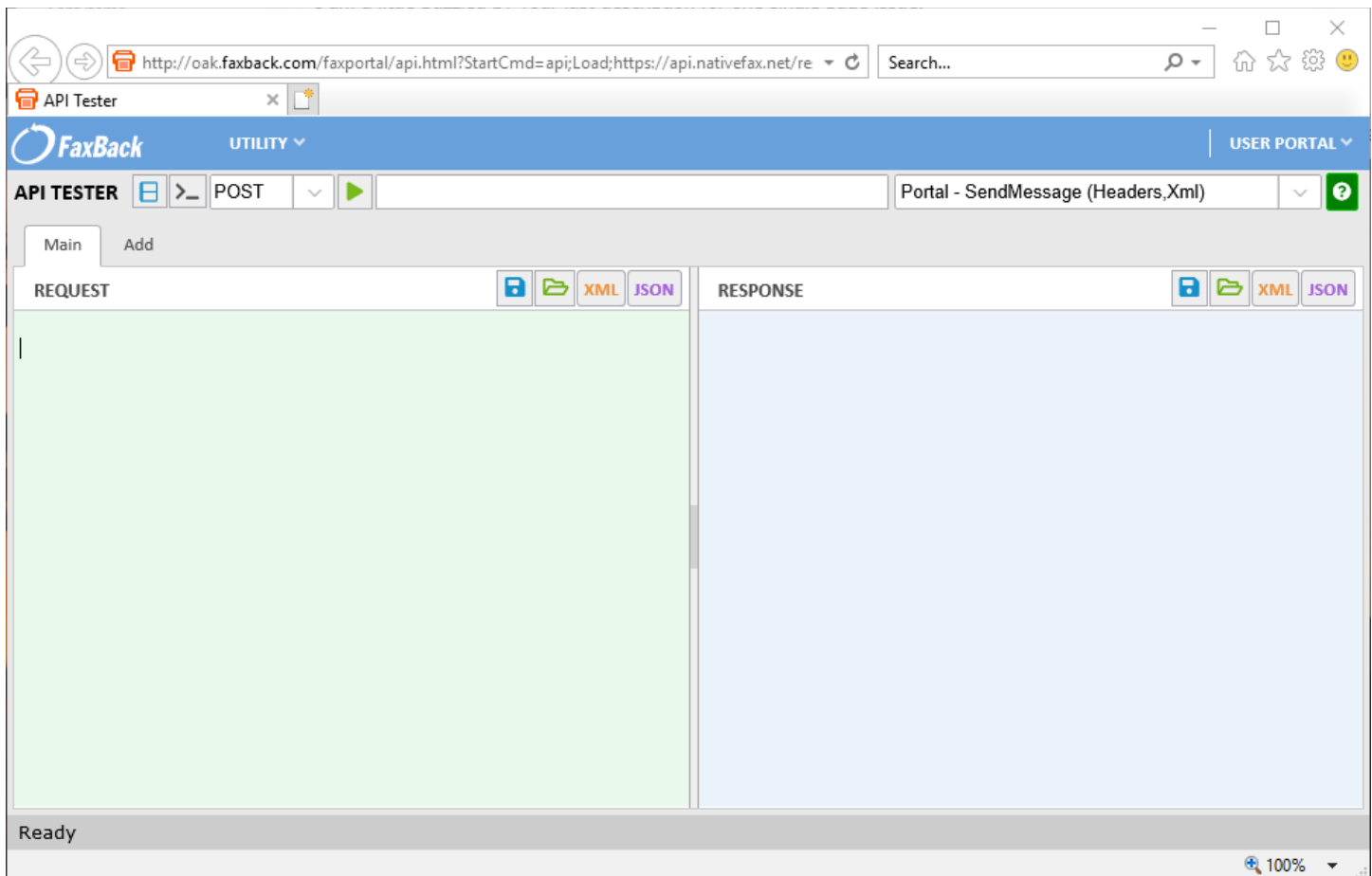
Group Object References

Property	Description	Json Example

API Tester

The FaxBack API tester is a simple browser application designed to provide a simple interface for testing Web Service API calls. The API tester communicates directly from the browser client the target web service. This model requires that the service it communicates with supports and is configured for CORS cross domain communication. If the target you would like to test does not support CORS then you will need to create an HTTP proxy target to redirect communications.

Try it: [API Tester - with NativeFax Samples](#)



Features Supported

- Send requests via the following HTTP/HTTPS methods: POST, GET, DELETE.
- Run javascript.
- Load single request from URL.
- Load sample set from URL and select sample from drop down.
- Load / save sample requests and results from local disk.
- Multi-tab interface allowing many tests to be managed at once.
- Convert requests from XML to/from JSON.
- Horizontal or vertical display orientation.
- Support single or multiple chained request / response sequences.
- Conversions supported:
 - encodeURIComponent and decodeURIComponent string.
 - Encode and decode string to base 64.

- Encode and decode FaxBack PW2 scramble (not encryption).
- Encode FaxBack NS9 password scramble (not reversible).
- Read file from local disk to base 64 string.
- Write file to local disk from base 64 string.

Main Toolbar

	Data conversion dialog
	Change between vertical and horizontal presentation.
POST 	Select action to be performed on run.
	Run specified action.
	URL to be used for action if not overridden by request content.
< select > 	Load sample.
	Help dialog

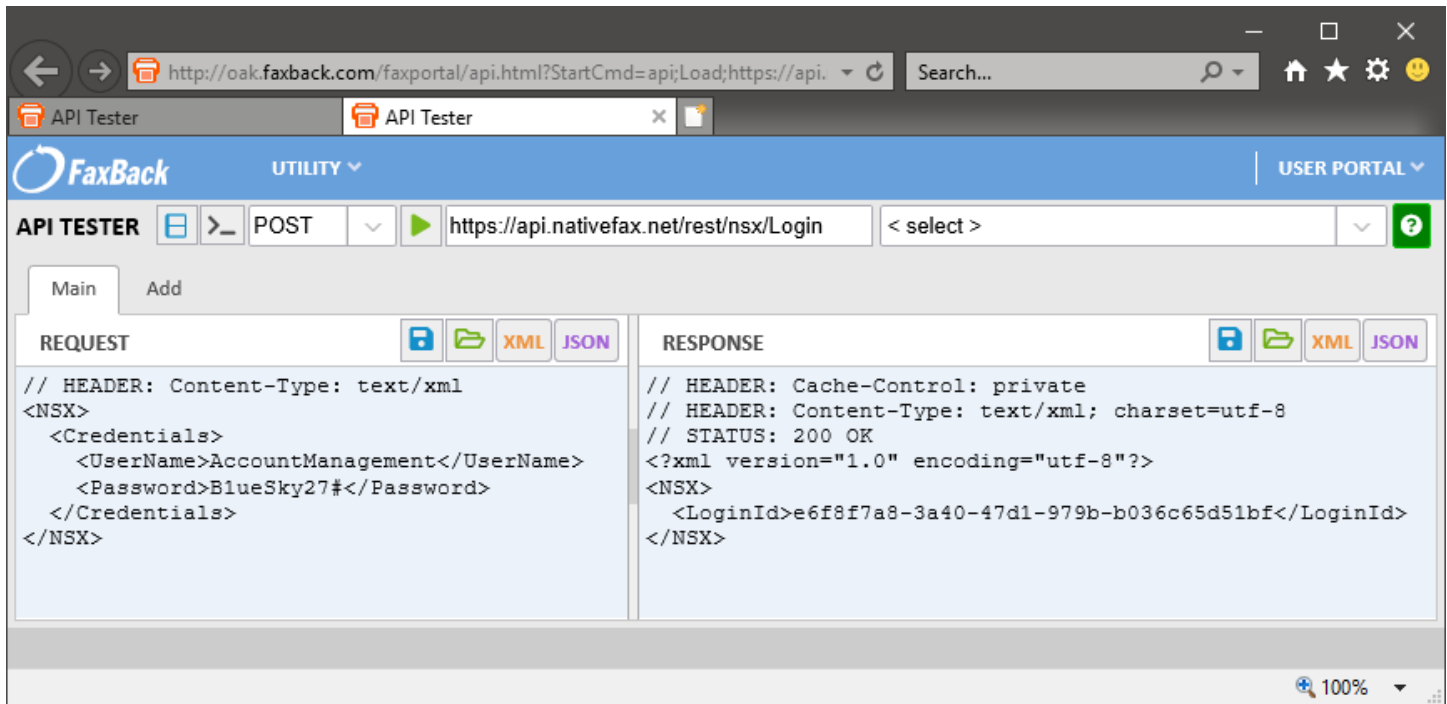
Request/Response Toolbar

	Save selected request or response text to disk file.
	Load selected request or response text from disk file.
	Convert selected text window to XML or friendly reformat XML if already XML.
	Convert selected text window to JSON or friendly reformat JSON if already JSON.

Single Request

The quickest way to use the API tester is with single request/response. Edit or paste a request into REQUEST editor and add a HEADER to specify the Content-Type to be transmitted. In this scenario the URL in the toolbar will be used for a POST, GET or DELETE operation.

Pressing the  run button will submit the request to the URL and display the output headers, status and response text in the RESPONSE edit field.



The screenshot displays the FaxBack API Tester application. The browser address bar shows the URL `http://oak.faxback.com/faxportal/api.html?StartCmd=api;Load;https://api.`. The application window has a title bar with "API Tester" and a search bar. The main interface features a blue header with the FaxBack logo, a "UTILITY" dropdown, and a "USER PORTAL" link. Below the header, the "API TESTER" section includes a toolbar with a "POST" method selected and a URL `https://api.nativefax.net/rest/nsx/Login`. The "REQUEST" editor shows an XML payload:

```
// HEADER: Content-Type: text/xml
<NSX>
  <Credentials>
    <UserName>AccountManagement</UserName>
    <Password>BlueSky27#</Password>
  </Credentials>
</NSX>
```

The "RESPONSE" editor shows the received XML data:

```
// HEADER: Cache-Control: private
// HEADER: Content-Type: text/xml; charset=utf-8
// STATUS: 200 OK
<?xml version="1.0" encoding="utf-8"?>
<NSX>
  <LoginId>e6f8f7a8-3a40-47d1-979b-b036c65d51bf</LoginId>
</NSX>
```

The interface also includes tabs for "Main" and "Add", and a status bar at the bottom indicating a zoom level of 100%.

Multiple Request

Multiple requests can be submitted in sequence with replacements fields being able to extract properties from previous requests by name and path.

The following inline settings are supported:

//	Comment
// NAME:	Sets name for subsequent response value lookup
// URL:	Service URL
// METHOD:	HTTP Request Type (GET/POST)
// HEADER:	HTTP Header to be sent (Content-Type: application/json)
//-----	Request separator, allows sending multiple requests at once
\${Name.Path}	Allows replacing content from one request from the result of another named request.

The screenshot shows the FaxBack API Tester interface. The top bar includes the FaxBack logo, a 'UTILITY' dropdown, and a 'USER PORTAL' dropdown. Below this, there's a 'Main' tab and an 'Add' button. The main area is divided into two panels: 'REQUEST' and 'RESPONSE'. The 'REQUEST' panel shows a sequence of requests: a POST request to 'https://api.nativefax.net/rest/nsx/Login' with XML credentials, followed by a GET request to 'https://api.nativefax.net/rest/nsx/Logout?LoginId=\${LogIn.NSX.LoginId}'. The 'RESPONSE' panel shows the corresponding responses: a 200 OK status for the login request with XML data including a 'LoginId', and a 204 No Content status for the logout request. The interface also features a search bar at the top right and a status bar at the bottom right showing '100%' zoom.